future internet

MDPI

Article

Calibration Granularity, Not Contamination: Diagnosing a TCN Anomaly Detector's False Positive Advantage in Cross-Dataset IoT Traffic

Muhammad Nouman *, Muhsin Hassanu * and Raja Ujjan

Department of Computing and Cybersecurity, University of the West of Scotland, London Campus, London E14 2BA, UK

* Correspondence: b01654699@studentmail.uws.ac.uk (M.N.); muhsin.hassanu@uws.ac.uk (M.H.)

Abstract

We set out to fix a "contamination" problem in reconstruction-based Temporal Convolutional Network VAEs (TCN-VAEs) for cross-dataset IoT flow anomaly detection: when attack flows share an encoder window with benign flows, the shared latent code is allegedly distorted, inflating benign reconstruction error and producing false positive rates (FPRs) of 22–65% despite an ROC-AUC above 0.93. Our proposed fix, TCN-Pred, excludes the target flow from the encoder and scores it by next-flow prediction error, reducing FPR to 0.65–13%. We subjected this causal explanation to a battery of controlled ablations, holding architecture, decoder, loss, and thresholding fixed while varying one factor at a time. Each one falsified the original hypothesis: target inclusion/masking changes FPR by at most 0.001; context shuffling/reversing/zeroing changes it by at most 0.003; a context-blind constant-output predictor matches TCN-Pred's FPR and F1 to three decimal places on all three datasets. The actual cause, confirmed on the original trained models with no retraining, is a scoring-granularity mismatch: the TCN-VAE threshold is calibrated from per-window errors averaged over 20 flows but applied to per-flow errors at evaluation (standard deviation $\approx \sqrt{20}\times$ higher, measured ratio 4.46 against a predicted 4.47). Recalibrating the identical model at matching granularity drops FPR from 22.7/47.6/64.6% to 0.65/5.0/12.5% on BoT-IoT, IoT-23 and ToN-IoT, closing 89–97% of the reported FPR gap without changing a single model weight. We report this diagnostic chain, together with an attack-prevalence sensitivity analysis, sample-disjoint calibration, normality diagnostics, and label-free and redundancy-aware (mRMR) feature-selection benchmarks, as a methodology other work should apply before attributing fixed-threshold performance to architecture. The pipeline is *supervised source-domain feature selection followed by benign-only detector training*, not fully unsupervised, a distinction we quantify later in the paper. Investigating dataset representativeness, we found that all three provided files reduce to only ≈6000 genuinely distinct flows via an undocumented row-duplication procedure, causing 97.8% BoT-IoT train/test near-duplicate overlap; a leakage-free re-evaluation changes FPR by only 0.23 percentage points. We also found that the TLS-metadata columns are already transformed upstream of every available artefact, so the proportion of genuinely TLS-encrypted flows cannot be recovered, and we soften the paper's encrypted-traffic framing accordingly.

Keywords: IoT network traffic; anomaly detection; temporal convolutional network; threshold calibration; ablation study; posterior collapse; variational autoencoder; cross-dataset generalisation; network intrusion detection

Check for updates

Received: 2 July 2026
Revised: 5 August 2026
Accepted: 13 August 2026
Published: 24 August 2026

Copyright: © 2026 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license.

1. Introduction

Modern IoT deployments almost universally encrypt device communications using Transport Layer Security (TLS), which rules out payload inspection as a detection primitive. Intrusion detection systems (IDSs) must instead work from flow-level statistics and TLS metadata alone [1,2]. This has pushed the field towards unsupervised anomaly detectors that learn a model of normal traffic and score each new flow against it [3]. We flag one caveat on this framing immediately, rather than only in the Methods or Limitations: every detector in this paper is trained on benign flows alone, but the feature-selection stage that picks which flow statistics those detectors see is fitted using BoT-IoT *attack* labels (ANOVA–Mutual Information; Section 3). The overall pipeline is, therefore, *supervised source-domain feature selection followed by benign-only detector training*, not a fully unsupervised system, and Section 4.20 quantifies how much of this paper's reported cross-dataset performance depends on that labelled stage rather than on the benign-only detector itself.

The operational consequences of detection errors are asymmetric: false negatives allow intrusions to proceed undetected, while false positive benign flows that are incorrectly flagged as attacks generate spurious alerts that consume analyst time and erode confidence in the system. In large-scale IoT deployments, even a 5% false positive rate can produce thousands of unnecessary alerts per day, making FPR a critical operational metric alongside ranking measures such as ROC-AUC.

The β-Variational Autoencoder (β-VAE) is a competitive choice for this setting: trained only on benign flows, it flags any flow whose reconstruction error exceeds a fixed threshold, giving strong anomaly-ranking performance across heterogeneous IoT datasets without retraining or threshold recalibration [4,5]. A natural extension is to give the detector access to surrounding flows rather than scoring each one in isolation, since IoT devices often follow regular communication schedules. Temporal Convolutional Networks (TCNs) [6] are a natural architecture for this: dilated causal convolutions— *causal* here means exclusively the left-to-right, past-only receptive field of a causal convolution in the signal-processing sense used by Bai et al. [6], not causal inference in the statistical sense, where information leakage is prevented from future time steps, giving a large receptive field relative to parameter count. Combining a TCN encoder with a VAE reconstruction objective (encode a window of flows, reconstruct the same window, and score each flow by its own reconstruction error) appears to unite sequence modelling with threshold-friendly probabilistic scoring.

1.1. The Hypothesis We Set Out to Test

Our starting hypothesis, and the original motivation for this paper, was that this reconstruction-based TCN-VAE suffers a *contamination problem*: if an encoder window contains both benign and attack flows, the shared latent vector z must reconcile their conflicting statistics, distorting the representation of the benign context and inflating the reconstruction error of co-occurring benign flows. We measured this empirically—a TCN-VAE trained on benign BoT-IoT flows and evaluated cross-dataset achieves an FPR of 22%, 48%, and 65% on BoT-IoT, IoT-23, and ToN-IoT, respectively, despite an ROC-AUC above 0.93 on all three (Section 3.5)—and proposed a fix: TCN-Pred, a causal next-flow predictor that excludes the target flow from the encoder and scores it by prediction error instead of reconstruction error. Under a cross-dataset no-recalibration protocol, TCN-Pred reduces FPR to 0.65–13%. In the original single-seed comparison, TCN-Pred produced F1 scores within 0.006 of the previously reported per-flow β-VAE baseline. We subsequently re-ran both model families over seeds 42–46 under this repository's current split; their mean F1 and FPR remain close on all three datasets (Section 4.10), providing a direct multi-seed replication of the original descriptive similarity. At this point, the paper's story seemed

complete: causal exclusion of the target flow prevents contamination, and a predictive objective is, therefore, preferable to a reconstructive one for this task.

This attribution is not safe as stated: TCN-Pred and TCN-VAE differ in several respects simultaneously (target inclusion, decoder output shape, window overlap, loss, and score distribution), so the FPR gap cannot be attributed specifically to target exclusion without a controlled ablation that holds everything else fixed. Section 4 reports five such ablations in sequence, each designed in response to what the previous one ruled out; every one falsified a plausible-sounding mechanism, until a sixth identified the actual cause—a scoring-granularity mismatch in threshold calibration, a statistical artefact rather than a property of the contamination story we started with. We report this diagnostic chain because the same mismatch can silently affect any fixed-threshold detector that aggregates scores differently at calibration time than at deployment time, and because the controls themselves (target-exposure ablation, context-permutation controls, decoder-shape ablation, and training-density ablation) are directly reusable by other researchers proposing a "temporal" or "causal" anomaly detector.

1.2. What the Ablations Showed

Table 1 previews the chain of evidence developed in Section 4. Each row holds architecture, decoder, loss, and the thresholding rule fixed and varies exactly one factor; each, in turn, failed to reproduce the TCN-VAE/TCN-Pred FPR gap, until the scoring-granularity mismatch (last row) closed it using the two original, already-trained models with no retraining at all.

Table 1. Summary of the five controlled ablations that preceded the correct diagnosis. Each varies one factor while holding architecture, decoder, loss, and the $\tau = \mu + 3\sigma$ thresholding rule fixed. FPR values are the mean across BoT-IoT, IoT-23, and ToN-IoT for the two conditions compared.

Ablation	What Was Varied/Finding	Verdict
Target exposure (Section 4.3)	Target flow excluded/included/masked in the encoder: FPR 0.070 vs. 0.071 vs. 0.071 (mean of 3 datasets).	Target exclusion is **not** the mechanism.
Context permutation (Section 4.4)	Real/shuffled/reversed/zeroed/mean context on the trained TCN-Pred: FPR within 0.003 of each other, including the zeroed condition.	Context content/order is **not** used.
Context-blind baseline (Section 4.5)	An unconditional constant predictor (no context at all) matches TCN-Pred's FPR/F1/AUC to 3 decimal places.	TCN-Pred behaves like a **prototype scorer**.
Decoder output shape (Section 4.6)	Single-flow decoder vs. whole-window reconstruction decoder, both scored per-flow, stride-1: FPR $\approx$ 0.06 for both.	Decoder shape is **not** the mechanism.
Training window density (Section 4.6)	TCN-VAE retrained with dense (stride-1, $\sim$13,580 windows) vs. canonical (stride-20, $\sim$680 windows) training data, both evaluated identically: FPR 0.222 vs. 0.227 (mean), stable across 5 seeds (std $\leq$ 0.006).	Training density is **not** the mechanism.
Scoring granularity (Section 4.7)	Same trained TCN-VAE, threshold recalibrated from per-flow instead of per-window benign errors (no retraining): FPR 22.7/47.6/64.6% $\rightarrow$ 0.65/5.0/12.5%.	**This is the mechanism.**

Real deployments add a second constraint that motivated the original design regardless of which mechanism turns out to explain it: the detector must hold up on networks it has never seen. A threshold calibrated on lab traffic will over-flag a factory floor without a recalibration step, and running that recalibration is impractical on embedded gateway

hardware. We therefore keep the cross-dataset no-recalibration protocol throughout this paper: every model is trained once on benign BoT-IoT flows, and the same fixed threshold is carried unchanged to IoT-23 (real-world Raspberry Pi IoT malware captures) and ToN-IoT (industrial IIoT sensor traffic), two environments with no overlap with the training domain.

1.3. Contributions

This paper makes the following contributions:

1. We identify and quantify a scoring-granularity mismatch in fixed-threshold calibration ($\tau = \mu + k\sigma$): calibrating on a statistic aggregated over more samples than the one used at evaluation time silently deflates the threshold and can, by itself, produce the FPR profile normally attributed to a "contamination" or architectural weakness. Recalibrating the granularity alone, on the original trained model, closes 22.7–64.6% FPR down to 0.65–12.5% (Section 4.7).
2. We provide a reusable five-part ablation battery—target-exposure ablation, context-permutation controls, a context-blind baseline, decoder-shape ablation, and training-density ablation—that isolates architectural and protocol factors one at a time, and we show all five are necessary: none alone would have found the true cause, and skipping straight to an architectural explanation would have been wrong (Sections 4.3–4.6).
3. We show that a plausible-looking causal predictor (TCN-Pred) can achieve strong cross-dataset FPR while being behaviourally indistinguishable from a context-blind constant predictor, cautioning against inferring genuine temporal or causal modelling from aggregate detection metrics alone (Section 4.5).
4. We reconstruct the attack-prevalence experiment properly—inserting contiguous attack episodes into a benign backbone and recomputing every context and prediction from scratch at each prevalence level, rather than reweighting already-computed scores—and confirm that FPR genuinely remains stable across 1–50% attack prevalence under this stronger test (Section 4.14).
5. We add a sample-disjoint calibration partition with normality diagnostics, a clean-calibration adaptive-thresholding comparison, a label-free feature-selection comparison, and a TLS-metadata ablation, addressing reproducibility and supervised-selection concerns (Sections 4.11–4.21).
6. We evaluate TCN-Pred cross-dataset (BoT-IoT $\rightarrow$ IoT-23, ToN-IoT) under a no-recalibration protocol: the source-domain threshold, fitted once on BoT-IoT, is applied unchanged to both target datasets, with no target-domain statistics used to adjust it (Section 3).
7. We report multi-seed reliability (5 seeds) with bootstrap confidence intervals for the TCN-family models, the recalibrated TCN-VAE, and the principal feature-selection comparison, and we disclose a data-provenance limitation: the shared, anonymised per-dataset files carry no timestamp, capture, or device identifier, which bounds what can be verified about the sequence-construction protocol (Sections 3.2 and 4.10).
8. We transparently investigate limitations of the dataset artefacts and the feature-selection pipeline: a forensic row-duplication and train/test-leakage analysis of the provided dataset files (Section 3.3), a correlation/redundancy analysis and a redundancy-aware multivariate (mRMR) benchmark of the ANOVA–MI feature selection (Sections 4.15 and 4.17), and a bootstrap stability analysis of the selected feature set (Section 4.18).

2. Related Work

2.1. Autoencoder-Based Anomaly Detection for IoT Traffic

Training an autoencoder on benign-only traffic and flagging high-reconstruction-error flows has become a standard starting point for unsupervised IoT anomaly detec-

tion [7]. Hou et al. [8] combined autoencoders with Bayesian Gaussian mixture models to improve scoring stability. Ayad et al. [9] extended these ideas using a one-class autoencoder combined with a deep neural network for real-time IoT anomaly detection; Berraadi et al. [10] addressed similar deployment constraints in IoT traffic management. Deterministic autoencoders are limited by point-estimate reconstructions that can yield poorly calibrated anomaly scores under distributional shift [11]. Generative approaches combining VAEs with adversarial training have also been explored for intrusion detection [12], though these require paired training data and are not directly applicable to the unsupervised, benign-only setting considered here.

Variational Autoencoders address this by learning a probabilistic latent space. The β-VAE [13] strengthens latent regularisation by scaling the Kullback–Leibler divergence term, promoting more disentangled representations. Lopez-Martin et al. [4] demonstrated that probabilistic modelling improves anomaly separability in IoT intrusion detection. A prior study from our group showed that a β-VAE with hybrid ANOVA–Mutual Information feature selection achieves strong cross-dataset generalisation on BoT-IoT, IoT-23, and ToN-IoT without retraining [5], establishing the per-flow baseline used in this work. None of this prior work examines whether the fixed threshold used for cross-dataset transfer is calibrated at the same score-aggregation granularity used at evaluation time—the question this paper turns out to be about.

2.2. Temporal Models for Network Intrusion Detection

Recurrent architectures such as LSTMs and GRUs have been applied to network intrusion detection to exploit sequential patterns in traffic [14]. Owoh et al. [15] explored temporal convolutional autoencoders for malicious-data detection in mobile crowd-sensing, demonstrating their capacity to capture sequential dependencies in sensor-data settings; however, that work does not evaluate on encrypted IoT network flows or under cross-dataset conditions.

A plausible-sounding failure mode motivated our original design: when the anomalous target flow is included in the encoder input window of a reconstruction-based detector, attack statistics could contaminate the shared latent representation, inflating false positive rates on mixed-class traffic sequences. The general principle that target inclusion can bias a reconstruction-based detector is not new to this paper—Hundman et al. [16] report a related benefit of predictive scoring over reconstruction for spacecraft telemetry, and forecasting-based detectors such as DeepAnT [17] are motivated by a similar concern in the general time-series literature. Our controlled ablations (Section 4.3) show this mechanism does *not* explain the FPR gap for TCN network-flow models; the real explanation is a calibration-granularity artefact (Section 4.7), illustrating how a plausible architectural mechanism can survive a first check and still fail systematic controls.

2.3. Temporal Convolutional Networks

TCNs were formalised by Bai et al. [6] as a general-purpose sequence modelling architecture combining dilated causal convolutions with residual connections. Causal padding ensures that the convolution output at time step t depends only on inputs at steps $t' \leq t$, preventing future leakage. Exponentially increasing dilation factors grow the receptive field to $(k-1)(2^n-1)+1$ with n residual blocks of kernel size k. Within network security, TCNs and related deep temporal architectures have been surveyed in the context of AI-based anomaly detection over encrypted traffic [18], and temporal convolutional autoencoders have been explored in sensor-data anomaly-detection settings [15]; to our knowledge, no prior work has applied TCN-based causal prediction to encrypted IoT

network flows under a cross-dataset no-recalibration protocol, nor has prior work reported the scoring-granularity mismatch diagnosed here.

2.4. Masked and Forecasting-Based Reconstruction; Score Calibration and Posterior Collapse

Beyond the LSTM-predictor and TCN-autoencoder work already discussed, the broader time-series anomaly-detection literature has explored several related directions not previously connected to encrypted IoT flow detection in this paper. Forecasting-based scoring, where a model predicts the next observation and scores by prediction error rather than reconstruction error, underlies DeepAnT [17], which reports a similar motivation to the one used here at a general time-series level, independent of this work. Masked or partially-blinded reconstruction, where part of the input window is withheld from the encoder and the model is scored on how well it recovers the withheld part, appears in sequence-to-sequence anomaly detectors such as OmniAnomaly [19] and USAD [20]; our masked-target-encoder control (Section 4.3) is a minimal instance of this idea, applied here to isolate target exposure rather than as a full detector architecture.

Posterior collapse—the phenomenon in which a VAE's approximate posterior converges to the prior, driving $D_{\mathrm{KL}} \to 0$—was first widely documented in sequence VAEs by Bowman et al. [21], and its interaction with the ELBO objective was analysed more precisely by Lucas et al. [22], who show that a linearised VAE can exhibit posterior collapse even when the decoder retains useful capacity conditioned on reparameterisation noise—a finding directly relevant to our own collapsed-KL result (Section 4.8) and, we now believe, to why TCN-Pred's predictions turned out to be near context-invariant (Section 4.5): with $D_{\mathrm{KL}} \approx 0$, the latent code carries almost no information about the context, so the decoder's output cannot depend strongly on it either. On threshold calibration, conformal prediction methods provide a principled alternative to a Gaussian tail rule for setting an anomaly threshold with a distribution-free coverage guarantee [23], and adaptive conformal inference extends this guarantee under distribution shift [24]—directly relevant to the cross-dataset setting studied here, where the calibration (BoT-IoT) and deployment (IoT-23, ToN-IoT) distributions differ, and, we now suggest, to the granularity question raised in this paper: a distribution-free calibration procedure would not by construction assume a particular aggregation level for the calibration statistic. We implement a split-conformal threshold (Section 4.12) alongside the empirical-quantile and Gaussian comparisons in Section 4.11: it achieves its nominal coverage bound in-domain but, as expected under covariate shift, not cross-dataset, which is, itself, informative about the limits of a naive (non-adaptive) conformal procedure in this setting and motivates the adaptive variant of Gibbs and Candès [24] as future work (Section 5.6).

2.5. Cross-Dataset Evaluation

Most IoT intrusion detection papers train and test on traffic from the same network environment [11], a practice that reliably produces strong numbers but says little about how the model behaves elsewhere. Widely used single-domain benchmarks such as CICIDS2017 [25] have driven substantial progress within one network environment, but in-domain accuracy, such a benchmark does not, by itself, predict behaviour on a structurally different network. Cantone et al. [11] showed empirically that classifiers achieving near-perfect accuracy in-domain often degrade substantially on traffic from a structurally different network: IoT flow statistics—flow duration, packet rates, and protocol mix—shift considerably between consumer home devices, real-world malware captures on heterogeneous hardware, and industrial IIoT sensor networks. A detection threshold calibrated on source-domain benign errors will over-flag target-domain traffic when the benign distribution drifts rightward under this domain shift, producing elevated false positive rates

without any change to the model—and, as this paper shows, a threshold miscalibrated in *aggregation granularity* rather than in *domain* can produce an almost identical symptom, making the two explanations easy to conflate.

Cross-dataset evaluation—training on one dataset, testing on another without retraining or threshold recalibration—provides a more realistic measure of generalisation. The protocol adopted in this work maintains an identical threshold derived from BoT-IoT benign training errors across all target datasets, consistent with the approach used in [5]. This no-recalibration constraint reflects the operational scenario where a deployed IoT gateway cannot access or label target-domain benign traffic to recalibrate its threshold after installation.

Table 2 summarises representative studies. Unlike this work, none of them tests whether a reported architectural improvement survives a battery of exposure, permutation, decoder-shape, and calibration-granularity controls.

Table 2. Summary of representative studies in temporal and autoencoder-based IoT anomaly detection. This is a small, representative comparison set, not an exhaustive survey.

Study	Method	Temporal?	Cross-Dataset?	Key Limitation
Nouman et al. (2026) [5]	β-VAE	No	Yes	Per-flow only; ignores temporal context
Owoh et al. (2024) [15]	Temporal AE	Yes	No	Single domain; no calibration-granularity check
Wang et al. (2024) [12]	VAE-GAN	No	No	Generative model; requires labelled attack data
Alaghbari et al. (2023) [7]	AE	No	No	No cross-dataset evaluation
Hundman et al. (2018) [16]	LSTM predictor	Yes	No	Non-network domain; no IoT evaluation
Lopez-Martin et al. (2017) [4]	Cond. VAE	No	No	Not evaluated on encrypted traffic
Bai et al. (2018) [6]	TCN	Yes	—	Architecture paper; no anomaly detection

3. Materials and Methods

3.1. Datasets

Three publicly available IoT intrusion detection benchmarks are used, following the evaluation protocol established in [5]. Table 3 summarises the dataset statistics.

Table 3. Balanced experimental subsets used in this study. Counts refer to the sampled and balanced partitions, not to the full raw dataset sizes. All subsets are processed using Zeek and CICFlowMeter; ANOVA–MI feature selection is fitted on BoT-IoT training data only and applied unchanged to all other partitions.

Dataset	Sampled Flows	Benign	Attack	Balance	Role
BoT-IoT (train)	27,200	13,600	13,600	50/50	Predictor training (benign-only)
BoT-IoT (test)	6800	3400	3400	50/50	In-dataset evaluation
IoT-23	6600	3300	3300	50/50	Cross-dataset evaluation
ToN-IoT	6600	3300	3300	50/50	Cross-dataset evaluation

The counts in Table 3 refer to the sampled and balanced experimental subsets used in this study; the original BoT-IoT, IoT-23, and ToN-IoT datasets are substantially larger in their raw form. As detailed in Section 3.3, these nominal row counts substantially overstate the number of independent observations: a forensic duplicate-cluster analysis shows each of the three provided files reduces to only ≈6000 genuinely distinct underlying flows, each represented on average 5.5–5.7 times via a documented row-duplication-with-Gaussian-noise procedure. We report this limitation in full rather than silently retain the larger nominal counts as if they denoted independent flows.

Terminology used throughout this paper. Because chronology, capture composition, and source-population representativeness cannot be verified from the artefacts available to us (Sections 3.2 and 3.4), we use "BoT-IoT," "IoT-23," and "ToN-IoT" from this point on as shorthand for the specific processed, balanced experimental subsets summarised in

Table 3—not as a claim that these subsets are verified, chronologically ordered, or representative samples of the original published datasets [26–28].

BoT-IoT [26] contains synthetic IoT traffic covering DDoS, reconnaissance, and data-exfiltration attacks. It is used exclusively for training; 13,600 benign flows construct the predictor's training set, and 6800 balanced test flows are used for in-dataset evaluation.

IoT-23 [27] contains real-world IoT malware captures from heterogeneous devices communicating over encrypted protocols, serving as the first cross-dataset evaluation target.

ToN-IoT [28] represents a complex IIoT environment with noisy and heterogeneous industrial traffic, serving as the harder second cross-dataset target.

All three datasets are preprocessed using Zeek (Version 6.0; The Zeek Project, https://zeek.org) [29] and CICFlowMeter to extract a unified set of 30 flow-level and TLS-aware metadata features including JA3/JA3S fingerprints, cypher suite identifiers, packet-length statistics, inter-arrival times, and flow duration metrics [30]. Hybrid ANOVA–Mutual Information (ANOVA–MI) feature selection [5,31,32] retains the top 20 features using BoT-IoT training labels; selected indices are applied unchanged to all datasets, preserving the no-recalibration protocol. The detector training itself is benign-only; however, the ANOVA–MI feature-selection stage uses source-domain BoT-IoT class labels, so the pipeline as a whole is *supervised source-domain feature selection followed by benign-only detector training*, not a fully unsupervised system. Section 4.20 quantifies how much of the reported cross-dataset performance this labelled stage contributes, using two label-free alternatives fitted without any class information. Features are normalised using min–max scaling fitted on BoT-IoT training data and applied frozen to all target datasets. Although source-domain min–max parameters were applied unchanged to IoT-23 and ToN-IoT, we did not retain a feature-wise audit of target-domain values falling outside the BoT-IoT scaling range. We therefore cannot quantify the percentage of out-of-range target values or determine whether individual shifted features disproportionately contribute to prediction MSE. This remains a limitation of the available processed artefacts.

Categorical feature encoding. The intended/documented preprocessing represents JA3 and JA3S fingerprints by frequency encoding and applies analogous source-fitted encoding to TLS version and cypher-suite identifiers. However, this encoding cannot be independently verified from the available artefacts: these four fields were supplied to this study as already transformed continuous numerical columns, and no upstream transformation code or raw categorical values survive. We therefore use the supplied numerical columns as received and do not claim to have fitted or reproduced their categorical encoding. Flag counts (SYN, FIN, RST) are treated as numerical features. The verifiable scaling and feature-selection operations in this study are fitted on BoT-IoT training data and applied unchanged to IoT-23 and ToN-IoT. Section 4.19 documents this provenance limitation and its consequences for claims about TLS coverage.

Table 4 lists the 20 selected features by category.

Table 4. Feature categories of the top-20 features selected by the ANOVA–MI pipeline. Selection is fitted on BoT-IoT training data and applied unchanged to all datasets. Exact column indices in rank order are listed in Appendix A.

Feature Category	Representative Features
Flow duration and timing	Flow duration, inter-arrival time mean and std
Packet volume statistics	Total forward/backward packets, total bytes
Packet length statistics	Max/min/mean packet length, packet length variance
Flow rate statistics	Packets per second, bytes per second
TLS metadata	TLS version and cypher suite
Flag counts	SYN, FIN, RST flag counts

3.2. Sequence Construction and Data Provenance

The sliding-window sequence construction requires explicit specification, since an under-specified construction could create artificial adjacency if, for example, attack flows were filtered from a chronological stream and the remaining benign flows were then treated as temporally consecutive. We give the exact algorithm used and are explicit about what can and cannot be verified from the artefacts available to us.

What is fully specified and reproducible? Within this study, sequences are built by a single deterministic operation on a flow-by-feature array X of shape (N, d) in the row order of the provided per-dataset file. No window crosses a file boundary because each dataset is processed as a single array; no additional concatenation of separate capture files is performed inside this pipeline. Two distinct windowing schedules are used across the models compared in this paper, and *the reported metrics for the two schedules are not computed over an identical subset of test flows*; Section 3.8 and Table 8 give the exact per-dataset flow counts, and Section 4.7 shows that this difference in flow coverage is not what drives the reported FPR gap.

Stride-1 (TCN-Pred, TCN-Det, and decoder-shape ablation variant A, Section 4.6). Training pairs are $(X[t-W:t],\ X[t])$ for $t \in [W, N)$, stride 1 (Equation (2), Section 3.6); evaluation windows are built the same way over the full test array, and the first W positions of every dataset (60 flows total across the three test sets, 20 per dataset) are excluded from all metrics via a Boolean valid mask, since these flows have no preceding context.

*Non-overlapping, stride = W (TCN-VAE's canonical protocol, Section 3.5, and the threshold-granularity recalibration check, Section 4.7).*The test array is instead partitioned into consecutive, non-overlapping windows of size W starting at position 0; every window is self-contained and requires no preceding context, so no flow is excluded and all N_{test} flows in every dataset receive a score. All three test-set sizes used in this study (6800/6600/6600) are exact multiples of $W = 20$, so this partition has no incomplete final window.

What cannot be verified from the available artefacts? The three per-dataset CSVs consumed by this pipeline already arrive as de-identified, pre-balanced 50%/50% flow tables: all source-IP, destination-IP, timestamp, capture-file, and device identifiers were stripped, and the class balance was fixed, at a data-preparation stage that precedes this study's codebase and predates the artefacts we have access to. Concretely, *every* downstream file we can inspect already has an exact 50/50 benign/attack split, and none carries a timestamp, capture, device, or scenario column. As a direct consequence we cannot independently confirm (i) whether the row order of the provided files reflects original chronological capture order, (ii) whether benign flows were extracted by filtering attacks from a mixed chronological stream (which would create false adjacency between originally non-consecutive benign flows), (iii) how multiple PCAP files, devices, or attack scenarios were concatenated upstream, or (iv) whether the BoT-IoT train/test split used to build Table 3 is capture-level, device-level, or flow-level. We regard this as a genuine limitation of the shared artefacts rather than a property we can respecify away, and we return to it in the Limitations (Section 5).

How the experiments in this paper are designed around that limitation. Rather than asserting that temporal adjacency in the provided row order is meaningful, several of the controlled experiments in this paper are constructed specifically so that their conclusions do not depend on that assumption holding: the context-permutation controls (Section 4.4) directly test whether the model's score is sensitive to the order and content of the context, using only the row order actually available; the calibration partition (Section 4.11) separates model-fitting flows from threshold-calibration flows by contiguous, non-overlapping row blocks rather than by claiming a verified chronological split; and the target-exposure ablation (Section 4.3) isolates the effect of target-flow encoder exposure while holding the

(unverified) context ordering fixed across all three variants compared, so any residual artefact of row order affects all three equally and cancels out of the comparison. As it turns out (Section 4.7), the effect we originally attributed to context/target exposure was not present at all once scoring-granularity was controlled, so this precaution, while still good practice, was not itself the deciding factor.

3.3. Row-Level Duplication in the Provided Dataset Files

Investigating whether the aggressive downsampling to the sizes in Table 3 yields a representative selected subset led to a more specific and more serious finding about the artefacts themselves, which we report in full below.

What we found. The repository history shows that the BoT-IoT raw-data file contained 6000 rows at an earlier commit and 34,000 rows in the next tracked commit. That commit also added the dataset-merging script, with targets of 34,000/33,000/33,000 rows for BoT-IoT/IoT-23/ToN-IoT respectively. That script's `expand_dataset()` function resamples existing rows *with replacement* (separately per class), adds independent $\mathcal{N}(0, 0.01^2)$ Gaussian noise to every added copy's 30 features, shuffles, and overwrites the CSV in place. We confirmed this forensically rather than relying on the commit history alone: clustering all rows of each raw CSV with DBSCAN ($\epsilon = 0.15$, chosen because the expected L_2 norm of 30-dimensional $\mathcal{N}(0, 0.01^2)$ noise is $0.01\sqrt{30} \approx 0.055$, an order of magnitude below the ≈3.4–6.2 typical distance between genuinely distinct rows) collapses every one of the three raw files to **exactly 6000 duplicate clusters**: BoT-IoT's 34,000 rows reduce to 6000 clusters (mean 5.67 copies/cluster, largest cluster 23 copies, 376 singleton/unique rows); IoT-23's 33,000 rows and ToN-IoT's 33,000 rows each also reduce to exactly 6000 clusters. No cluster mixes benign and attack labels internally, confirming duplicates preserve their source row's label. In short: each of the three datasets used in this study contains only **≈6000 genuinely distinct underlying flows**, each represented on average 5.5–5.7 times via noise-perturbed copies; the nominal row counts in Table 3 substantially overstate the number of independent observations.

Consequence for the BoT-IoT in-dataset train/test split. `data_preprocessing.py` applies `sklearn.train_test_split(test_size=0.2, random_state=42, stratify=y)` to the already-duplicated 34,000-row table with no awareness of the duplicate-cluster structure. Re-deriving that exact split and cross-referencing it against the DBSCAN clusters shows that 3767 of the 6000 clusters (62.8%) have members in *both* the training and test partitions, and **97.8% of BoT-IoT test rows have a near-duplicate (differing only by the $\sigma = 0.01$ jitter) in the training set**. This is a train/test leakage mechanism the original pipeline did not account for, and it is a threat to every BoT-IoT *in-dataset* metric reported anywhere in this paper's tables. It does *not* affect the IoT-23/ToN-IoT *cross-dataset* numbers, which come from separate source files never split against BoT-IoT's training data.

Quantifying the effect. We cannot undo the underlying ≈6000-flow ceiling (the genuine-representativeness question remains unanswerable from the artefacts available to us; Section 3.4), but we can remove the specific train/test-split leakage above. We built a cluster-aware 80/20 split that assigns every duplicate cluster wholly to train *or* test (never both), class-stratified at the cluster level, and retrained TCN-Pred identically (same harness, hyperparameters, seed 42) on it. Table 5 compares the results against the canonical (leaky-split) BoT-IoT numbers reported throughout this paper.

What this changes and what it does not. The row-duplication ceiling means BoT-IoT's "13,600 benign training flows" (Table 3) should be read as, at most, a few thousand independent benign observations, heavily oversampled with small perturbations, not as 13,600 independently captured flows. We do not retroactively rerun every ablation in this paper against the deduplicated split; doing so is a substantial undertaking, and every

ablation in this paper compares multiple models trained and evaluated under the *same* leaky protocol, so relative comparisons between them (the paper's central diagnostic claims, e.g., the TCN-Pred vs. TCN-VAE FPR gap and its granularity-mismatch explanation in Section 4.7) are not invalidated by a leakage source applied identically to every compared model. What changes is how absolute BoT-IoT in-dataset numbers should be read: as an optimistic upper bound on this detector family's true generalisation to unseen BoT-IoT flows, not a clean estimate of it. The IoT-23 and ToN-IoT cross-dataset numbers, which this paper already treats as the harder and more meaningful test, are unaffected by this specific mechanism.

Table 5. BoT-IoT in-dataset TCN-Pred performance: canonical (leaky) split vs. a cluster-aware split with no near-duplicate leakage between train and test.

Split	ROC-AUC	PR-AUC	F1	Prec.	Recall	FPR
Canonical (leaky, as reported)	0.9996	0.9996	0.9913	0.9935	0.9891	0.0065
Cluster-aware (leakage-free)	0.9997	0.9996	0.9901	0.9911	0.9891	0.0088

Cluster-aware split: 27,192 train/6808 test rows, same TCN-Pred architecture/hyperparameters/seed = 42 as the canonical protocol. Despite 97.8% of the canonical test set having a near duplicate in canonical training, removing that leakage changes FPR by only 0.0023 and F1 by 0.0012—both well within this paper's measured seed-to-seed variation (Section 4.10). We read this as evidence that this specific detector (benign-only training, fixed-threshold anomaly scoring, never trained to discriminate specific attack vs. benign pairs) does not strongly exploit this leakage, not as evidence that the leakage is unimportant in general; a discriminatively trained classifier evaluated on the same split would be at materially higher risk from it.

3.4. Subsampling Representativeness

The aggressive downsampling to 13,600 benign BoT-IoT training flows requires justification: ideally, a demonstration that the selected subset is representative of the full source dataset (e.g., attack-type coverage, protocol mix) rather than a convenient slice. We cannot do this. The only artefacts available to us are `data/raw/{bot_iot,iot23,ton_iot}.csv`, and each already arrives at exactly the row count reported in Table 3 (34,000/33,000/33,000 rows respectively)—there is no larger source file, no record of which flows from the original BoT-IoT/IoT-23/ToN-IoT releases were retained or discarded, and no protocol or attack-category column, and (Section 3.3) the row-inflation script that produced the current file sizes is a documented *duplication-with-noise* procedure operating on an already-reduced 6000-row base, not a sampling procedure operating on the full original datasets. We therefore cannot compare the selected subset's attack-family distribution, protocol distribution, or flow-duration/byte-count/packet-count distributions against the true source population because that population is not present in, and cannot be reconstructed from, the artefacts we have access to. We do not describe this experimental subset as representative of the full BoT-IoT, IoT-23, or ToN-IoT datasets; it is the available experimental subset, and Table 3 and the Limitations (Section 5.6) are worded accordingly. The computational justification for working at this scale is straightforward: it is the scale we were given, not a scale we chose for computational convenience—but that is a different claim from representativeness, and we do not conflate the two.

3.5. The Hypothesised Contamination Problem in Reconstruction-Based TCN-VAEs

Let $\mathbf{X} = [x_1, x_2, \ldots, x_N]$ denote a sequence of N network flows, each $x_i \in \mathbb{R}^d$. A reconstruction-based TCN-VAE encodes the window $[x_{t-W+1}, \ldots, x_t]$ to produce a latent distribution $q(z \mid \mathbf{X}_{t-W:t})$ and decodes z to reconstruct the full window. The anomaly score of flow x_t is the reconstruction error of that flow within its window. Figure 1 illustrates the hypothesised contamination mechanism this scoring procedure motivates, contrasted with TCN-Pred's causal-exclusion design.

Scoring and aggregation procedure. The evaluation sequence is partitioned into *non-overlapping* windows of size W (stride $= W$), so every flow belongs to exactly one window. The encoder consumes the whole W-flow window—including whichever flows in it happen to be attacks—and produces one shared z per window; the decoder then reconstructs all W flows from that single z, and every flow in the window receives its own per-feature reconstruction MSE as its individual score.

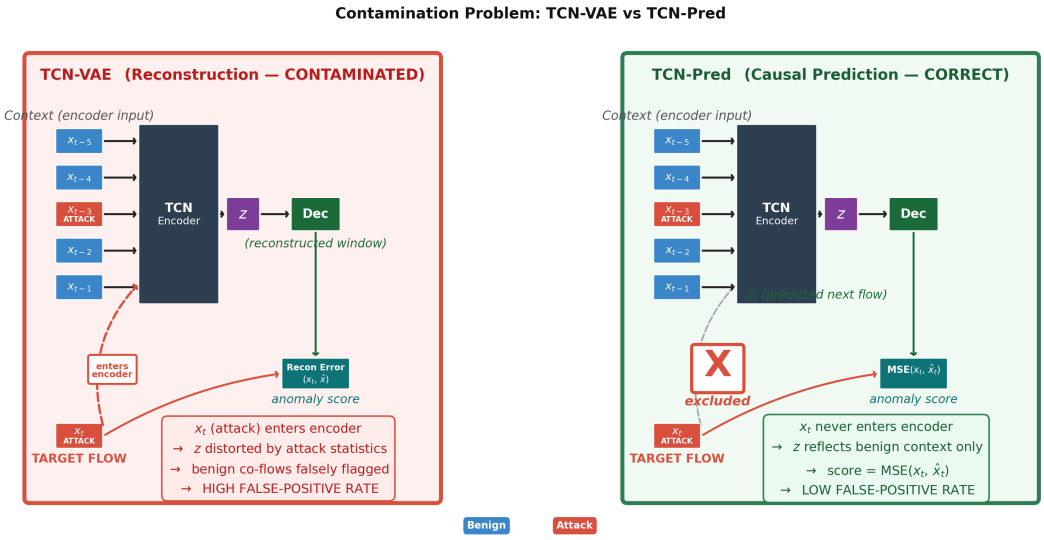


Figure 1. Illustration of the contamination hypothesis we originally proposed and subsequently falsified (Section 4.7). (**Left**): In the TCN-VAE, attack flows (red) share an encoder window with benign flows (blue), which we hypothesised would distort the shared latent vector z and inflate the reconstruction error of benign co-occurring flows. (**Right**): In TCN-Pred, the target flow x_t is excluded from the encoder; we hypothesised this design would avoid the effect shown on the left. Section 4.7 shows that the actual FPR gap between these two models is explained by a threshold-calibration granularity mismatch rather than by the contamination mechanism illustrated here.

In the 50% benign/50% attack evaluation protocol used in this study, assuming independent 50/50 class assignment per position, the probability that a window of size W contains at least one attack flow is $1 - 0.5^W$. For $W = 20$, this equals $1 - 0.5^{20} \approx 1.0$, so virtually every window presented to the encoder during evaluation contains at least one attack flow. This observation motivated our original contamination hypothesis; Table 6 reports the FPR gap that motivated it, and Section 4.7 reports the controlled diagnosis that explains it.

Table 6. The FPR gap that originally motivated the contamination hypothesis. TCN-VAE uses reconstruction scoring with a per-window-calibrated threshold (the canonical protocol defined in Section 3.7); TCN-Pred uses causal prediction. **Bold** indicates best result per column. See Table 17 for what happens when TCN-VAE's threshold is recalibrated at matching granularity.

Model	ROC-AUC			FPR		
	BoT-IoT	**IoT-23**	**ToN-IoT**	**BoT-IoT**	**IoT-23**	**ToN-IoT**
TCN-VAE (window-calibrated threshold)	0.9987	0.9818	0.9317	0.2218	0.4773	0.6500
TCN-Pred	**0.9996**	**0.9879**	**0.9471**	**0.0065**	**0.0560**	**0.1299**

Bold values indicate the better result per column. ROC-AUC is threshold-independent and is essentially unaffected by the calibration-granularity question; only FPR and other threshold-dependent metrics are affected.

3.6. Model Architectures

Both models compared in Section 3.5 share the same TCN encoder family; they differ in what the encoder sees and what the decoder outputs.

TCN-VAE (reconstruction). Encodes a W-flow window (including the flow being scored) and reconstructs the entire window from one shared latent code:

$$\begin{aligned}(\mu, \log\sigma^2) &= \text{TCNEncoder}(\mathbf{X}_{t-W+1:t}), \qquad z = \mu + \epsilon \odot \sigma, \\ \hat{\mathbf{X}} &= \text{Decoder}_{\text{window}}(z) \in \mathbb{R}^{W\times d}.\end{aligned} \tag{1}$$

trained with $\mathcal{L} = \text{MSE}(\mathbf{X}, \hat{\mathbf{X}}) + \beta D_{\text{KL}}(q(z \mid \mathbf{X}) \| \mathcal{N}(0, I))$.

TCN-Pred (causal prediction). Encodes only the W flows *preceding* the target and predicts a single flow vector,

$$(\mu, \log\sigma^2) = \text{TCNEncoder}(\mathbf{C}_t), \quad \mathbf{C}_t = [x_{t-W}, \ldots, x_{t-1}] \tag{2}$$

$$z = \mu + \epsilon \odot \sigma, \quad \epsilon \sim \mathcal{N}(0, I), \qquad \hat{x}_t = \text{Decoder}_{\text{single}}(z) \in \mathbb{R}^d \tag{3}$$

$$s_t = \frac{1}{d}\sum_{j=1}^{d}(x_{t,j} - \hat{x}_{t,j})^2 \tag{4}$$

trained with $\mathcal{L} = \text{MSE}(x_t, \hat{x}_t) + \beta D_{\text{KL}}(q(z \mid \mathbf{C}_t) \| \mathcal{N}(0, I))$.

TCN encoder (shared). Three residual TCN blocks with channel widths $[64, 64, 32]$ and kernel size $k = 3$; dilation factors $1, 2, 4$ yield a receptive field of $(k-1)(2^3-1)+1 = 15$ flows. A final average-pooling over the time dimension followed by two linear layers produces $\mu \in \mathbb{R}^{16}$ and $\log\sigma^2 \in \mathbb{R}^{16}$.

Decoders. $\text{Decoder}_{\text{single}}$: $\text{Linear}(16 \to 128) \to \text{ReLU} \to \text{Linear}(128 \to d)$. $\text{Decoder}_{\text{window}}$: identical hidden layer, final layer $\text{Linear}(128 \to Wd)$ reshaped to (W, d). Both use dec_hidden $= \text{channels}[-1] \times 4 = 128$.

Table 7 summarises the shared hyperparameters.

3.7. Training and Threshold Derivation

Both models are trained exclusively on benign BoT-IoT flows. TCN-Pred uses a stride-1 sliding window over 13,600 benign training flows, producing $13{,}600 - W = 13{,}580$ (context, target) pairs. TCN-VAE uses non-overlapping windows (stride $= W$), producing 680 training windows—the canonical TCN-VAE protocol used throughout this paper; Section 4.6 tests whether this difference in training-set size explains the FPR gap (it does not).

After training, a fixed anomaly threshold is derived from the benign training error distribution:

$$\tau = \mu_{\text{train}} + 3\sigma_{\text{train}} \tag{5}$$

This is where the mechanism we diagnose in Section 4.7 originates. For TCN-Pred, μ_{train} and σ_{train} are computed over the 13,580 individual *per-flow* prediction errors s_t. For the canonical TCN-VAE protocol, μ_{train} and σ_{train} were, instead, computed over 680 *per-window* errors—each already an average over $W \times d = 400$ values (Equation (1) with the mean taken over both the window and feature axes) rather than over $d = 20$ values, as for TCN-Pred. Averaging over more values mechanically shrinks the standard deviation of the resulting statistic; the empirical consequence is quantified in Section 4.7. This threshold is applied unchanged to BoT-IoT test, IoT-23, and ToN-IoT; no target-domain error distribution is used for recalibration.

For TCN-Pred (and TCN-Det, Section 4.9), evaluation uses the same stride-1 construction as training: a sliding window is built over the full test sequence (benign and attack flows in their original interleaved order), and the first $W = 20$ flows have no context and are excluded from metric computation via a boolean valid mask (20 flows per dataset; 60 of 20,000 total test flows excluded), so metrics for these models are computed on $N_{\text{test}} - W$ flows per dataset. TCN-VAE's canonical evaluation (Table 6), instead, uses the

non-overlapping, stride-W partition described in Section 3.5: every one of the N_{test} flows in each dataset is scored, none excluded, because a non-overlapping window needs no preceding context. Both schedules report a *per-flow* score for every flow they cover—the same scoring granularity—so the two evaluations differ only in exactly which flows are covered (N_{test} vs. $N_{\text{test}} - W$, at most a 0.3% difference in flow count) in window overlap, not in whether the reported score is per-flow or per-window. Table 8 (Section 3.8) states the exact flow count underlying every reported metric. The genuine difference between the two models is upstream of evaluation regarding how τ was *calibrated* (previous paragraph): TCN-Pred's threshold comes from 13,580 per-flow training errors, TCN-VAE's canonical threshold from 680 per-window training errors. This calibration-granularity mismatch, not any difference in evaluation-time scoring or flow coverage, is the mechanism diagnosed in Section 4.7.

Table 7. Shared TCN-VAE/TCN-Pred architecture and training hyperparameters. Where applicable, training settings were aligned with the per-flow β-VAE baseline [5]; architecture-specific parameters (TCN channels, kernel size, dilation factors, latent dimension, and decoder structure) were selected independently.

Hyperparameter	Value
Context/window size W	20 flows
Input features d	20 (ANOVA–MI selected)
TCN channels	[64, 64, 32]
Kernel size k	3
Dilation factors	1, 2, 4
Receptive field	15 flows
Latent dimension	16
Decoder hidden size	128
Dropout	0.1
β (KL weight)	2.0
Optimiser	Adam
Learning rate	10^{-3}
Batch size	256
Epochs	50
TCN-Pred training samples	13,580 (context, target) pairs, stride 1
TCN-VAE training samples	680 non-overlapping windows, stride $= W$

3.8. Evaluation Protocol

All experiments follow a strict cross-dataset no-recalibration protocol: each model is trained once on BoT-IoT benign flows, and its fixed threshold τ is applied without modification to all three test sets. No target-domain data is used at any stage for model adaptation, feature re-selection, or threshold recalibration. **The windowing schedule differs by model family and is not interchangeable**: TCN-Pred, TCN-Det, and the LSTM/GRU/persistence/moving average/Unconditional mean baselines (Sections 4.5–4.9) use stride-1 windows with the first W flows per dataset excluded for lack of preceding context; the canonical TCN-VAE (Table 6) and the granularity-recalibration check (Table 17) use non-overlapping, stride-W windows with no exclusion, since every such window is self-contained. Table 8 states the exact number of flows each family's metrics are computed over for every dataset. The canonical results (Table 6 and the comprehensive comparison in Section 4.2) use a fixed random seed (42) for reproducibility, matching the protocol of [5]; Section 4.10 additionally reports a 5-seed reliability study. Primary evaluation metrics are ROC-AUC, PR-AUC (average precision), F1-score, precision, recall, and FPR, each computed at the per-flow level for every flow the given model's protocol covers.

Table 8. Exact number of test flows underlying each reported metric, by windowing schedule. All stride-1 models (TCN-Pred, TCN-Det, LSTM/GRU analogues, persistence, moving average, Unconditional mean, and decoder-shape variant A) share the middle column; the canonical TCN-VAE and the granularity-recalibration check (Table 17) use the right-hand column.

Dataset	Total Test Flows	Stride-1 (Excl. First W)	Non-Overlapping, Stride $= W$
BoT-IoT	6800	6780	6800
IoT-23	6600	6580	6600
ToN-IoT	6600	6580	6600

All three totals are exact multiples of $W = 20$, so the non-overlapping schedule has no incomplete final window. The stride-1 schedule excludes exactly 20 flows per dataset (60 of 20,000 total) for lack of preceding context.

3.9. The Ablation Battery

The FPR gap in Table 6 could reflect differences in decoder shape, loss, target exposure, training-set size, or calibration protocol rather than a genuine contamination mechanism. Rather than argue this in the abstract, we built one controlled experiment per candidate mechanism, holding everything except the manipulation under test fixed, and we ran them in the order below—each was designed in response to what the previous one ruled out.

(1) Target-exposure ablation (Section 4.3). Three variants of the identical TCN-Predictor class—same architecture, decoder, loss, window size $W = 20$, and thresholding rule—differ only in what the encoder's context window contains: *excluded* (context $= [x_{t-W}, \ldots, x_{t-1}]$, i.e., canonical TCN-Pred), *included* (context $= [x_{t-W+1}, \ldots, x_t]$, i.e., the target flow is the final frame the encoder sees), and *masked* (same window as *included*, final frame zeroed). We additionally stratify the canonical model's benign-target errors by the number of attack flows in the context window.

(2) Context-permutation controls (Section 4.4). Using the already-trained canonical TCN-Pred model, with its fixed threshold unchanged, we recompute prediction errors with the context manipulated at evaluation time only: *shuffled*, *reversed*, *zeroed*, and *mean*-replaced (BoT-IoT benign training mean).

(3) Expanded baseline coverage, including a context-blind control (Section 4.5). Under the identical protocol, we add: *persistence* ($\hat{x}_t = x_{t-1}$, a naive AR(1) baseline), *moving average* ($\hat{x}_t = \text{mean}(\text{context})$), *unconditional mean* (the context-blind, MSE-optimal constant predictor; the achievable ceiling for any architecture that ignores context entirely), and *LSTM/GRU* predictors that share TCN-Pred's variational head and single-vector decoder but replace the causal TCN encoder with a recurrent encoder.

(4) Decoder-shape and training-density ablations (Section 4.6). We built a hybrid model with a window-reconstruction decoder (TCN-VAE-style) but with stride-1 training/evaluation and target-included context, scored only at its final (target) position, isolating decoder output shape from window stride. We then re-evaluated that same model under non-overlapping, all-positions-shared-z scoring (identical to TCN-VAE's canonical protocol) to isolate window sharing. Finally, we retrained the actual `TCNVAE` class with stride-1 (dense, ∼13,580 windows) training data instead of the canonical stride-20 (∼680 windows) data, across 5 seeds, to isolate training-set size.

(5) Threshold-granularity diagnosis (Section 4.7): the result that these four controls converged on: we take the original, already-trained canonical TCN-VAE model with no modification whatsoever and recompute its threshold from *per-flow* instead of *per-window* benign training errors; then, we re-evaluate on all three datasets with the unchanged per-flow evaluation scoring already used for Table 6.

Beyond this battery, we also report: (6) multi-seed reliability with bootstrap confidence intervals (Section 4.10); (7) a sample-disjoint calibration partition with normality diagnostics (Section 4.11); (8) an adaptive-thresholding comparison with clean calibration

(Section 4.13); (9) a realistic, non-reweighted attack-prevalence reconstruction (Section 4.14); (10) a label-free feature-selection comparison (Section 4.20); and (11) a TLS-metadata-removed evaluation (Section 4.21).

4. Results

4.1. In-Dataset and Cross-Dataset Performance of TCN-Pred

Table 9 reports TCN-Pred's in-dataset performance on BoT-IoT, and Table 10 extends this to the full cross-dataset comparison. The ablation battery in Section 3.9 concerns *why* TCN-Pred out-performs TCN-VAE on FPR, not what TCN-Pred's own numbers are.

Table 9. TCN-Pred in-dataset performance on BoT-IoT (W = 20, $\beta = 2$, $\tau = 1.2883$). Evaluable flows: 6780 (first 20 excluded).

ROC-AUC	PR-AUC	F1	Precision	Recall	FPR	TN/FP/FN/TP
0.9996	0.9996	0.9913	0.9935	0.9891	0.0065	3368/22/37/3353

Table 10. Comprehensive performance comparison across all three datasets. β-VAE results from [5]. TCN-VAE uses the canonical window-calibrated threshold protocol (see Section 4.7 for the recalibrated version). TCN-Pred uses causal prediction. **Bold** indicates the best result per metric among β-VAE and TCN-Pred.

Dataset	Model	ROC-AUC	PR-AUC	F1	Prec.	Recall	FPR
BoT-IoT	β-VAE [5]	**0.9996**	**0.9996**	**0.9922**	**0.9938**	**0.9900**	**0.0062**
	TCN-VAE (window-calib.)	0.9987	0.9985	0.9000	0.8512	0.9552	0.2218
	TCN-Pred	**0.9996**	**0.9996**	0.9913	0.9935	0.9891	0.0065
IoT-23	β-VAE [5]	**0.9882**	**0.9877**	**0.9425**	0.9416	**0.9433**	0.0585
	TCN-VAE (window-calib.)	0.9818	0.9804	0.8069	0.6710	1.0000	0.4773
	TCN-Pred	0.9879	0.9874	0.9401	**0.9437**	0.9365	**0.0560**
ToN-IoT	β-VAE [5]	0.9465	0.9477	0.8732	0.8645	0.8821	0.1433
	TCN-VAE (window-calib.)	0.9317	0.9262	0.7523	0.6036	0.9991	0.6500
	TCN-Pred	**0.9471**	**0.9482**	**0.8788**	**0.8722**	**0.8855**	**0.1299**

Bold values indicate the best-performing model for each metric among β-VAE and TCN-Pred. TCN-VAE (window-calib.) is included to reproduce the comparison that originally motivated this paper; Section 4.7 shows its FPR is a calibration artefact rather than a property of the architecture.

Figures 2–6 visualise this same TCN-Pred comparison in more detail: per-flow error distributions, ROC and precision–recall curves, confusion matrices, and threshold calibration against the training distribution. Figure 7 additionally plots the raw anomaly-score trace across the first 600 flows of each dataset in original arrival order.

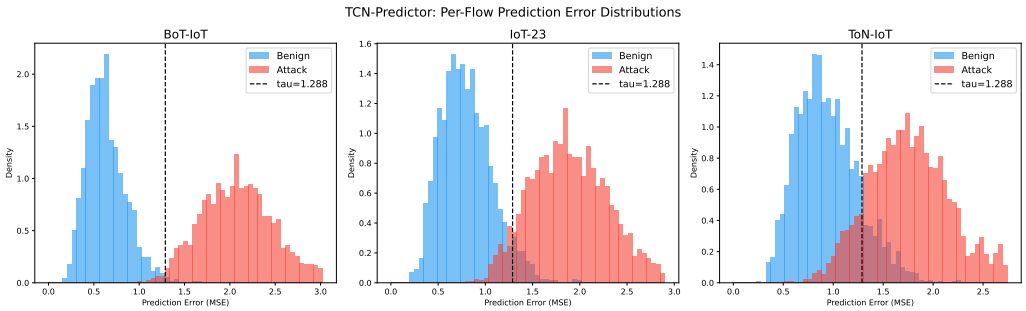


Figure 2. Per-flow prediction-error distributions for benign (blue) and attack (red) flows across all three datasets (left: BoT-IoT; centre: IoT-23; right: ToN-IoT). The dashed vertical line indicates the fixed threshold $\tau = \mu_{\text{train}} + 3\sigma_{\text{train}}$, derived from BoT-IoT benign training errors and applied without modification to all three datasets.

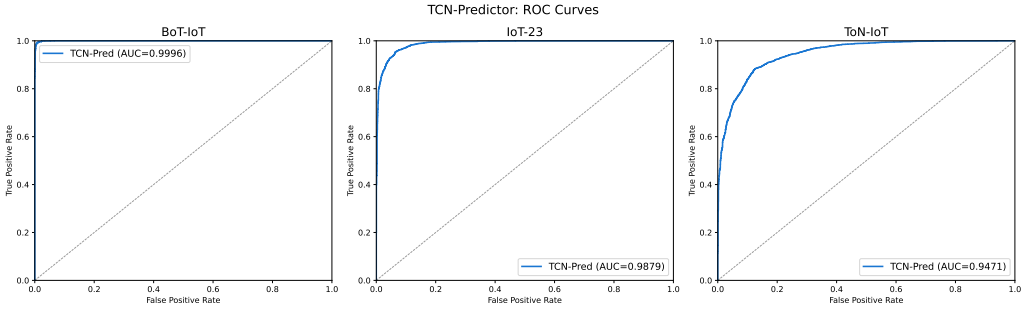


Figure 3. ROC curves for TCN-Pred on BoT-IoT (in-dataset, (**left**)), IoT-23 (cross-dataset, (**centre**)), and ToN-IoT (cross-dataset, (**right**)). ROC-AUC: 0.9996, 0.9879, and 0.9471 respectively. ROC-AUC is threshold-independent and is, therefore, unaffected by the calibration-granularity question examined in Section 4.7.

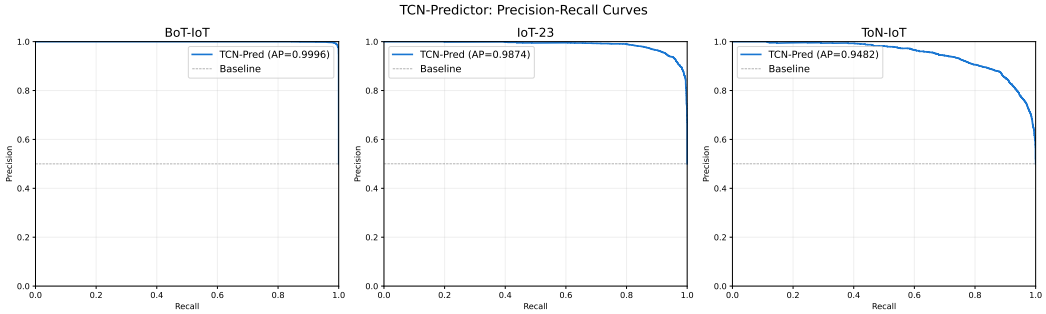


Figure 4. Precision–recall curves for TCN-Pred on BoT-IoT (**left**), IoT-23 (**centre**), and ToN-IoT (**right**). Average precision (AP): 0.9996, 0.9874, and 0.9482.

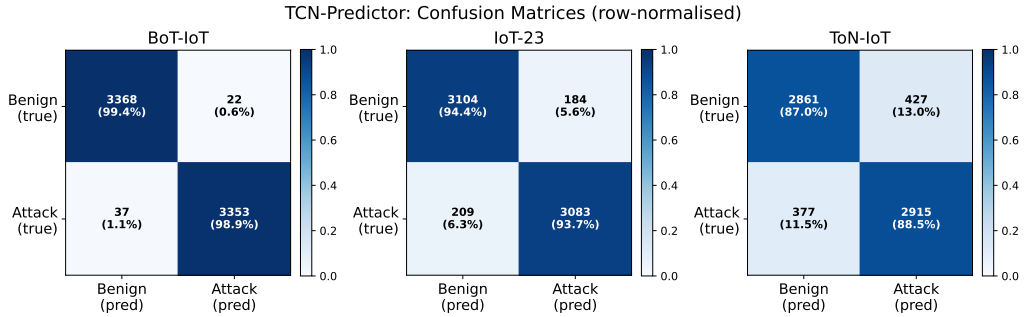


Figure 5. Row-normalised confusion matrices for TCN-Pred on BoT-IoT (**left**), IoT-23 (centre), and ToN-IoT (**right**).

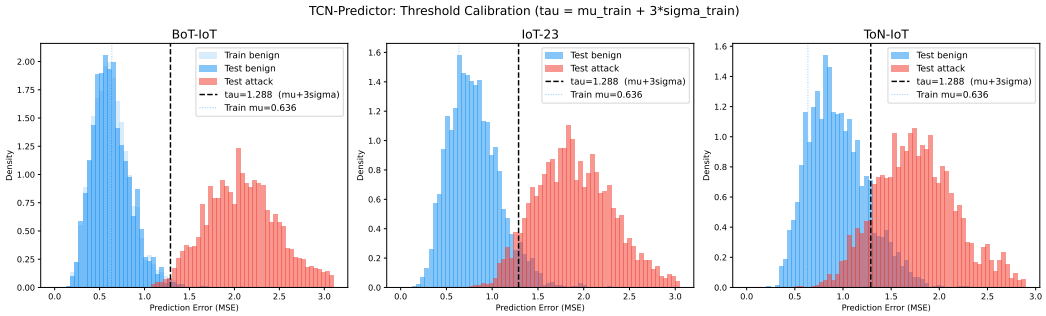


Figure 6. Threshold calibration across datasets (left: BoT-IoT; centre: IoT-23; right: ToN-IoT). Each panel overlays training benign (light blue), test benign (blue), and test attack (red) error distributions, with the fixed threshold τ marked.

On ToN-IoT, TCN-Pred achieves an ROC-AUC of 0.9471 and an F1 of 0.8788 with FPR 0.1299. The original single-seed comparison placed this within 0.006 F1 of the previously

reported per-flow β-VAE baseline [5]. We have now retrained both model families over seeds 42–46 under the current repository split (Table 22): their mean F1 differs by at most 0.001 across all three datasets, and their distributions overlap. This supports a descriptive conclusion of similar performance and provides no evidence of superiority for the temporal model; the 13–14% ToN-IoT FPR also remains operationally high.

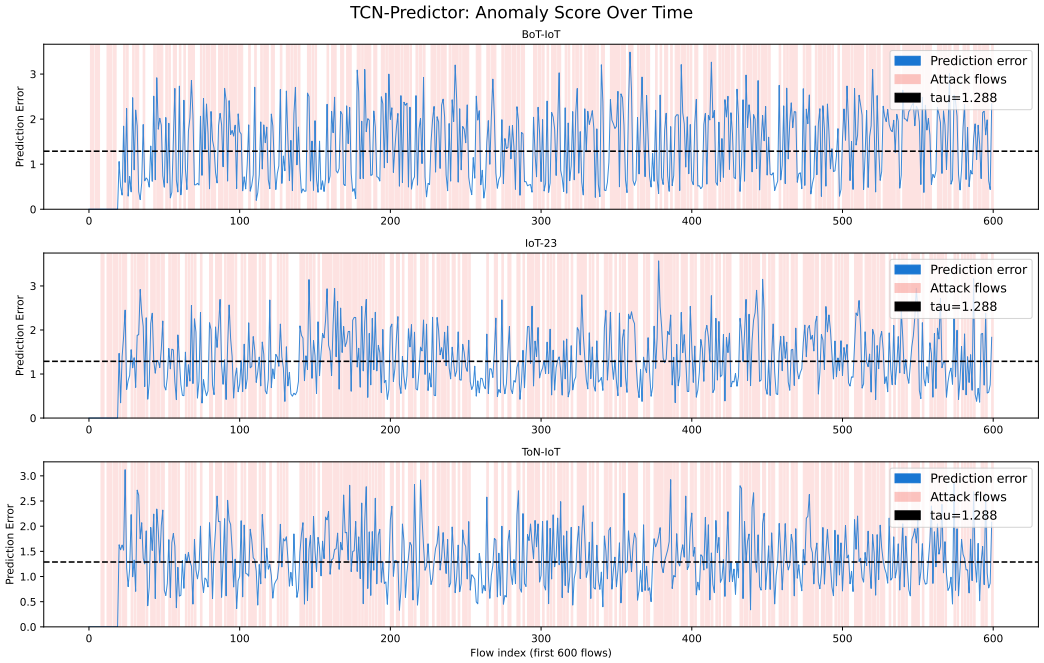


Figure 7. TCN-Pred anomaly score (prediction MSE) across the first 600 flows of each dataset in original arrival order. Section 4.5 shows that a context-blind constant predictor produces a visually similar trace, so this figure should be read as evidence that TCN-Pred separates classes, not as evidence that it uses temporal structure to do so.

4.2. Comprehensive Performance Comparison

Table 10 presents the comparison between the per-flow β-VAE, TCN-VAE under its original (window-calibrated) threshold, and TCN-Pred.

4.3. Target-Exposure Ablation

Table 11 reports the three target-exposure variants described in Section 3.9: identical architecture, decoder, loss, window size, and thresholding rule, varying only whether the target flow is excluded from, included in, or masked within the encoder's context window.

Table 11. Target-exposure ablation. All three variants share architecture, decoder, loss, and thresholding rule; only encoder exposure to the target flow x_t differs. Differences of this size ($\leq$0.001 FPR, $\leq$0.0003 F1) are within the seed-to-seed variance quantified in Section 4.10.

Dataset	Variant	ROC-AUC	PR-AUC	F1	Prec.	Recall	FPR
BoT-IoT	Excluded (canonical)	0.9996	0.9996	0.9907	0.9935	0.9879	0.0065
	Included	0.9996	0.9996	0.9905	0.9935	0.9876	0.0065
	Masked	0.9996	0.9996	0.9904	0.9935	0.9873	0.0065
IoT-23	Excluded (canonical)	0.9878	0.9873	0.9407	0.9448	0.9365	0.0547
	Included	0.9878	0.9874	0.9401	0.9440	0.9362	0.0557
	Masked	0.9878	0.9874	0.9402	0.9442	0.9362	0.0554
ToN-IoT	Excluded (canonical)	0.9470	0.9481	0.8759	0.8704	0.8815	0.1314
	Included	0.9471	0.9483	0.8759	0.8697	0.8821	0.1323
	Masked	0.9471	0.9483	0.8767	0.8704	0.8830	0.1317

Target exclusion changes the FPR by at most 0.001 and F1 by at most 0.0003 on every dataset. If target exclusion prevented contamination, as originally hypothesised, the *included* variant (where the encoder can see the exact value it must predict) should show a markedly higher FPR under mixed benign/attack evaluation; it does not. Table 12 stratifies the canonical (excluded) model's benign-target errors by the number of attack flows present in the surrounding context window.

Table 12. Canonical TCN-Pred: benign-target prediction error and flag rate, stratified by the number of attack flows present in the $W = 20$-flow context window. No clean-context (zero-attack) samples exist in the balanced 50/50 evaluation protocol.

Dataset	Attacks in Context	n	Mean Error	Flag Rate
BoT-IoT	1–2	3	0.582	0.000
	3–5	39	0.605	0.026
	6+	3348	0.634	0.006
IoT-23	1–2	1	0.752	0.000
	3–5	62	0.777	0.016
	6+	3225	0.802	0.056
ToN-IoT	1–2	1	0.908	0.000
	3–5	62	0.898	0.129
	6+	3225	0.943	0.132

Mean error rises only mildly (BoT-IoT: 0.605→0.634 from 3–5 to 6+ attacks; a 4.8% relative increase) as context attack count grows from a handful to the majority of the window, and flag rate does not increase monotonically with attack count on IoT-23/ToN-IoT. This is consistent with the context-permutation results (Table 13): the model's score is not strongly sensitive to context content.

4.4. Context-Permutation Controls

Table 13 reports the context-permutation controls: the already trained canonical TCN-Pred model and its fixed threshold are left completely unchanged; only the context fed to the encoder at evaluation time is manipulated.

Table 13. Context-permutation controls on the trained TCN-Pred model (threshold fixed at $\tau = 1.2883$ throughout). $|\Delta s|$ is the mean absolute score change for the same target flows relative to the original ordered context; flip rate is the fraction of flows whose classification (flagged/not flagged) changes.

Dataset	Context Condition	ROC-AUC	F1	FPR	$\lvert\Delta s\rvert$	Flip Rate
BoT-IoT	Original (ordered)	0.9996	0.9913	0.0059	–	–
	Shuffled	0.9996	0.9916	0.0056	0.0181	0.0015
	Reversed	0.9996	0.9908	0.0065	0.0177	0.0013
	Zeroed	0.9996	0.9916	0.0056	0.0181	0.0012
	Mean-replaced	0.9996	0.9911	0.0059	0.0179	0.0007
IoT-23	Original (ordered)	0.9879	0.9406	0.0544	–	–
	Shuffled	0.9879	0.9410	0.0557	0.0171	0.0046
	Reversed	0.9880	0.9401	0.0557	0.0173	0.0064
	Zeroed	0.9881	0.9404	0.0550	0.0176	0.0061
	Mean-replaced	0.9880	0.9393	0.0563	0.0173	0.0062
ToN-IoT	Original (ordered)	0.9471	0.8779	0.1293	–	–
	Shuffled	0.9474	0.8770	0.1314	0.0172	0.0102
	Reversed	0.9473	0.8771	0.1311	0.0167	0.0088
	Zeroed	0.9471	0.8763	0.1314	0.0169	0.0123
	Mean-replaced	0.9472	0.8763	0.1326	0.0174	0.0116

Even the *zeroed* condition—an all-zero context carrying no information whatsoever—performs within the 0.001–0.002 FPR and F1 of the real, ordered context on every dataset, with under 1.3% of flows changing classification. TCN-Pred does not use the content or order of its context in any way that measurably affects its score.

4.5. Expanded Baselines and a Context-Blind Control

Table 14 adds the baselines described in Section 3.9, evaluated under the identical protocol as TCN-Pred (same benign-only training, same $\tau = \mu + 3\sigma$ rule, and same three-dataset no-recalibration evaluation).

Table 14. Expanded baseline coverage. *Persistence* and *moving average* are genuine (untrained) autoregressive baselines; *Unconditional mean* is a context-blind, MSE-optimal constant predictor; *LSTM/GRU* share TCN-Pred's decoder and loss but replace the TCN encoder with a recurrent one. **Bold** marks TCN-Pred for reference.

Dataset	Model	ROC-AUC	F1	FPR
BoT-IoT	Persistence (x_{t-1})	0.5020	0.3650	0.2855
	Moving average	0.4962	0.2675	0.1714
	Unconditional mean (context-blind)	0.9996	0.9914	0.0059
	LSTM predictor	0.9996	0.9904	0.0071
	GRU predictor	0.9996	0.9911	0.0059
	TCN-Pred (canonical)	**0.9996**	**0.9913**	**0.0065**
IoT-23	Persistence (x_{t-1})	0.5107	0.3146	0.2141
	Moving average	0.5104	0.2587	0.1597
	Unconditional mean (context-blind)	0.9879	0.9389	0.0572
	LSTM predictor	0.9878	0.9410	0.0550
	GRU predictor	0.9878	0.9399	0.0563
	TCN-Pred (canonical)	**0.9879**	**0.9406**	**0.0544**
ToN-IoT	Persistence (x_{t-1})	0.5081	0.2818	0.1931
	Moving average	0.5095	0.2582	0.1661
	Unconditional mean (context-blind)	0.9466	0.8764	0.1341
	LSTM predictor	0.9473	0.8771	0.1314
	GRU predictor	0.9471	0.8753	0.1329
	TCN-Pred (canonical)	**0.9471**	**0.8779**	**0.1293**

Persistence and moving average—genuine autoregressive baselines—perform near chance (AUC $\approx$ 0.50–0.51), showing that individual IoT flow feature vectors are not smoothly autocorrelated at this granularity: naively extrapolating from the immediate past does not work. Yet the context-blind Unconditional mean predictor, the TCN, the LSTM, and the GRU all land within 0.001–0.002 FPR and F1 of each other. The encoder architecture is not the reason any of them work.

Two conclusions follow. First, per-flow feature vectors are not smoothly autocorrelated: persistence and moving average, the two baselines that would benefit from genuine short-range temporal structure, perform at chance level. Second, every learned architecture we tested—TCN, LSTM, GRU, and a model with no context at all—converges to statistically indistinguishable performance. Combined with the already-reported KL collapse ($D_{\mathrm{KL}} \approx 10^{-5}$; Section 4.24), this is consistent with all of these models learning close to the same thing: a context-independent estimate of the benign-flow distribution, scored by squared distance. The window-size ablation (flat from $W = 5$ to $W = 50$; Section 4.23) and the context-permutation controls (Section 4.4) are two more views of the same phenomenon.

4.6. Decoder-Shape and Training-Density Ablations

If target exposure and context content do not explain the FPR gap in Table 6, two architectural candidates remain: TCN-VAE's decoder reconstructs a whole window rather than predicting one flow, and TCN-VAE trains on 680 non-overlapping windows rather than TCN-Pred's 13,580 overlapping ones. Table 15 tests both, one at a time, on the identical window-reconstruction-decoder model.

Table 15. Decoder-shape ablation. The same trained window-reconstruction-decoder model (encoder, decoder, and loss modelled directly on TCN-VAE, but trained on stride-1 windows with the target as the final context frame) is evaluated two ways: (A) stride-1, scoring only the final (target) position of each flow's own dedicated window; (B) non-overlapping stride-W windows with every position scored from one shared reconstruction, which is identical to TCN-VAE's canonical evaluation protocol.

Dataset	Evaluation Protocol	ROC-AUC	F1	FPR
BoT-IoT	(A) stride-1, last-position only	0.9996	0.9916	0.0062
	(B) non-overlapping, all positions (TCN-VAE-style)	0.9996	0.9916	0.0059
IoT-23	(A) stride-1, last-position only	0.9884	0.9413	0.0575
	(B) non-overlapping, all positions (TCN-VAE-style)	0.9879	0.9391	0.0579

Table 15. *Cont.*

Dataset	Evaluation Protocol	ROC-AUC	F1	FPR
ToN-IoT	(A) stride-1, last-position only	0.9479	0.8785	0.1341
	(B) non-overlapping, all positions (TCN-VAE-style)	0.9465	0.8756	0.1339

Reconstructing the whole window instead of predicting one flow (decoder shape) and switching from a dedicated fresh window per flow to a shared, non-overlapping window (window sharing/stride) both leave FPR essentially unchanged ($\leq$0.003 across all six cells). Neither is the mechanism.

This still leaves training-set size: TCN-VAE's canonical protocol trains on 680 non-overlapping windows, roughly 20$\times$ fewer than the 13,580 overlapping windows used everywhere else in this paper. Table 16 retrains the actual `TCNVAE` class—not a hand-built analogue—with stride-1 (dense) training data instead of the canonical stride-20 data, evaluated under TCN-VAE's own canonical protocol, across 5 seeds.

Table 16. Training-density ablation. The real `TCNVAE` class, retrained with stride-1 (13,580-window) instead of the canonical stride-20 (680-window) benign training data, evaluated under the unchanged canonical (non-overlapping, all-positions) protocol. FPR reported as mean $\pm$ std over 5 seeds (42–46).

Dataset	Stride-20 (Canonical, 680 Windows)	Stride-1 (Dense, 13,580 Windows)
BoT-IoT	FPR = 0.2266 $\pm$ 0.0028	FPR = 0.2217 $\pm$ 0.0013
IoT-23	FPR = 0.4788 $\pm$ 0.0062	FPR = 0.4853 $\pm$ 0.0003
ToN-IoT	FPR = 0.6569 $\pm$ 0.0038	FPR = 0.6675 $\pm$ 0.0014

FPR is statistically indistinguishable between 680-window and 13,580-window training across 5 seeds each (differences $\leq$ 0.010, well within 1–2 standard deviations), and both remain far above TCN-Pred's FPR. Training-set size is not the mechanism. The very low seed-to-seed standard deviation ($\leq$0.006 in every cell) also rules out training-run variance/bimodality as an explanation: TCN-VAE's high FPR is a reproducible property of its evaluation protocol, not a matter of which local optimum a given run happens to reach.

4.7. Threshold-Granularity Diagnosis

None of the five preceding ablations moved the needle. The actual explanation was sitting in the two threshold files already saved from the original training runs. The canonical TCN-Pred threshold has a mean of 0.6358 and a standard deviation of 0.2175 ($\tau = 1.2883$); the canonical TCN-VAE threshold has a mean of 0.6478—almost identical—but a standard deviation of 0.0496, a factor of 4.4$\times$ smaller. Section 3.7 explains why: TCN-Pred's threshold is calibrated on 13,580 individual per-flow prediction errors, while TCN-VAE's threshold is calibrated on 680 per-*window* reconstruction errors, each already an average over all $W = 20$ flows and all $d = 20$ features in that window. Under independence (or only weak dependence) among the averaged values, this shrinks the standard deviation of the resulting statistic by a factor of approximately $\sqrt{W} = \sqrt{20} = 4.47$—matching the observed ratio of 4.46 almost exactly, which is consistent with this dataset's per-flow prediction errors being close to independent across the window. This is an empirical match for this feature space and model, not a claim that $\sqrt{W}$ holds universally; see Section 5.2 for the caveat when within-window errors are correlated.

Table 17 tests this directly: we take the identical, already-trained canonical TCN-VAE model (`tcn_vae_bot_iot.pt`, no retraining, no weight changes) and recompute τ from *per-flow* instead of *per-window* benign training errors, then re-evaluate on all three datasets using the same per-flow scoring function (`per_flow_reconstruction_errors`) already used to produce Table 6.

Multi-seed confirmation. Table 17 uses the single canonical seed-42 TCN-VAE. Since this is the paper's central finding, we retrained five independent TCN-VAE models from scratch (seeds 42–46, with full training run each time, not just recalibration of one fixed model) and computed both threshold calibrations for every seed. The effect reproduces closely: the window-calibrated FPR is 0.2269 $\pm$ 0.0030 (BoT-IoT), 0.4803 $\pm$ 0.0046 (IoT-23),

and 0.6550 ± 0.0024 (ToN-IoT) across the 5 seeds, while the per-flow recalibration brings this down to 0.0066 ± 0.0005, 0.0517 ± 0.0008, and 0.1310 ± 0.0035, respectively, matching the single-seed numbers in Table 17 to within the ordinary seed-to-seed variation reported elsewhere in this paper (Section 4.10). The measured std ratio (per-flow σ divided by per-window σ) is 4.51 ± 0.05 across the 5 seeds, consistent with the $\sqrt{20} = 4.47$ theoretical prediction in every case (`results/threshold_granularity_multiseed.json`). The granularity effect is, therefore, not an artefact of one particular random initialisation.

Table 17. Threshold -granularity diagnosis. Same trained TCN-VAE model, same per-flow evaluation scores, and two threshold calibrations. "Original" reproduces Table 6; "Recalibrated" uses the identical benign BoT-IoT training flows, scored per-flow instead of per-window, to derive τ.

Dataset	Threshold Calibration	μ	σ	τ	FPR
(training stats)	Original (per-window, $n = 680$)	0.6478	0.0496	0.7965	—
	Recalibrated (per-flow, $n = 13{,}600$)	0.6475	0.2212	1.3111	—
BoT-IoT	Original		$\tau = 0.7965$		**0.2271**
	Recalibrated		$\tau = 1.3111$		**0.0065**
IoT-23	Original		$\tau = 0.7965$		**0.4764**
	Recalibrated		$\tau = 1.3111$		**0.0497**
ToN-IoT	Original		$\tau = 0.7965$		**0.6455**
	Recalibrated		$\tau = 1.3111$		**0.1248**

ROC-AUC is unchanged by recalibration (0.9986/0.9809/0.9338, identical to the original-threshold row in each case) because it is threshold-independent; only FPR and other threshold-dependent metrics move. Recalibrating a single number, with no retraining, closes 89–97% of the original FPR gap with TCN-Pred on every dataset (e.g., BoT-IoT: from a 34.2× gap over TCN-Pred's 0.0065 down to parity).

We regard this as the paper's central finding. It means the entire diagnostic chain in Sections 4.3–4.6—five independent, carefully controlled experiments—was necessary precisely because the true cause was not any of the plausible architectural mechanisms that motivated the original design, including the one we set out to fix. We had no way to know this in advance: the granularity mismatch is easy to introduce (calibrating a threshold on whatever statistic a training-time evaluation function happens to return, without checking that it matches the deployment-time scoring function) and easy to miss (both models still report "reasonable" ROC-AUCs, so nothing looks obviously broken), which is why we report it as a general methodological hazard rather than a bug specific to our own earlier implementation.

4.8. Deterministic vs. Variational Predictor

A natural question, independent of the granularity diagnosis above, is whether the variational component contributes anything beyond a deterministic predictor when the Kullback–Leibler (KL) divergence collapses during training. We trained a deterministic TCN predictor (TCN-Det) with identical architecture, training data, window size, and threshold protocol, but without reparameterisation and without KL loss ($\mathcal{L} = \text{MSE}(x_t, \hat{x}_t)$ only). Table 18 reports the resulting comparison against variational TCN-Pred across all three datasets.

TCN-Det achieves near-perfect recall (0.997, 1.000, and 0.995) but an FPR of 7.1%, 22.9%, and 37.6%—markedly worse than TCN-Pred's 0.65–13%. Given the granularity diagnosis in Section 4.7, our initial hypothesis was that reparameterisation noise widens the benign-training-error distribution used to set $\tau = \mu + 3\sigma$, lowering FPR through the same statistical channel diagnosed there. The matched-noise experiment below tests this hypothesis directly and does not confirm it: fixed Gaussian noise at several magnitudes fails to reproduce TCN-Pred's σ_{train} or its FPR. The TCN-Det/TCN-Pred gap should, therefore, be read as only partly diagnosed, not as proof of a variational-inference benefit. Section 4.10's new five-seed sample-disjoint calibration check reduces TCN-Det's mean FPR

to 3.4/14.0/27.0% (from 6.2/19.6/35.7% under same-sample calibration), but it remains above TCN-Pred's 0.7/6.0/14.3%. Calibration optimism, therefore, explains part, but not all, of the gap; TCN-Det also remains substantially more seed-sensitive.

Table 18. Deterministic TCN-Det vs. variational TCN-Pred. Both models use identical architecture, training data, and threshold protocol ($\tau = \mu + 3\sigma$). **Bold** indicates best result per column.

Model	Dataset	ROC-AUC	F1	Prec.	Recall	FPR
TCN-Det (no VAE)	BoT-IoT	**0.9996**	0.9657	0.9299	0.9972	0.0711
	IoT-23	**0.9899**	0.8957	0.8053	1.0000	0.2293
	ToN-IoT	0.9513	0.8340	0.7237	0.9946	0.3756
TCN-Pred (VAE, ours)	BoT-IoT	**0.9996**	**0.9913**	**0.9935**	**0.9891**	**0.0065**
	IoT-23	0.9879	**0.9401**	**0.9437**	**0.9365**	**0.0560**
	ToN-IoT	**0.9471**	**0.8788**	**0.8722**	**0.8855**	**0.1299**

TCN-Det has high recall but an inflated FPR relative to TCN-Pred. Given Section 4.5's finding that TCN-Pred behaves close to a context-blind constant predictor, our initial hypothesis was that reparameterisation noise acts as a regulariser that widens the benign training error distribution (a wider σ_{train} mechanically raises τ; see Section 4.7). The matched-noise experiment immediately below tests this directly and does **not** confirm it—fixed Gaussian noise fails to reproduce either TCN-Pred's σ_{train} or its FPR—so this gap is reported as unexplained rather than attributed to a specific mechanism.

Testing the reparameterisation-noise hypothesis directly. The "wider σ_{train}" explanation above makes a specific, testable prediction: if it is noise injection *per se* that widens σ_{train} and lowers the FPR, then a deterministic encoder with *fixed*, non-learned Gaussian noise added to its latent code at a chosen standard deviation (no KL loss and no learned logvar) should show σ_{train} and FPR moving together as that noise magnitude increases. We tested noise_std $\in \{0, 0.05, 0.1, 0.2, 0.5\}$, with noise_std $= 0$ reproducing canonical TCN-Det exactly (Table 19).

Table 19. Deterministic TCN predictor with fixed, non-learned Gaussian latent noise at several magnitudes. noise_std $= 0$ is canonical TCN-Det. No KL loss is used at any noise magnitude.

noise_std	σ_{train}	FPR BoT-IoT	FPR IoT-23	FPR ToN-IoT
0.0 (=TCN-Det)	0.1950	0.0711	0.2293	0.3756
0.05	0.1975	0.0493	0.2141	0.3796
0.1	0.1972	0.0935	0.2868	0.4446
0.2	0.2024	0.1127	0.3054	0.4571
0.5	0.2116	0.0711	0.1946	0.3452

σ_{train} rises only slightly across this range (0.195 to 0.212)—well short of TCN-Pred's learned $\sigma_{\text{train}} = 0.217$–$0.221$ (Table 17; Section 4.20's calibration figures)—and FPR does *not* move monotonically with noise_std: it worsens at noise_std $= 0.1$–0.2 before improving at 0.5, and every configuration tested remains far above TCN-Pred's 0.65–13% range. **This does not confirm the reparameterisation-noise hypothesis as stated.** Fixed, non-learned noise injection alone is not sufficient to reproduce TCN-Pred's regularising effect; whatever TCN-Pred is doing depends on the learned, per-sample logvar and/or the KL term specifically, not on latent noise magnitude in isolation. We report this as a negative result and withdraw the "plausible explanation" framing from the previous paragraph accordingly; the TCN-Det/TCN-Pred FPR gap remains diagnosed only as a correlation with σ_{train}, not as a demonstrated causal mechanism.

4.9. Comparison with Classical Baselines

Table 20 places TCN-Pred and TCN-Det alongside the classical models deterministic Autoencoder (AE), Random Forest (RF), Support Vector Machine (SVM), and Isolation Forest (IF) evaluated on the same ANOVA–MI-selected feature space; classical baseline results are taken from [5].

Provenance note (self-containment). This paper's own TCN-Pred, TCN-Det, and recalibrated-TCN-VAE numbers are newly generated inside this study's codebase, on the BoT-IoT/IoT-23/ToN-IoT partitions and ANOVA–MI feature indices documented in Section 3 and Appendix A. The AE/RF/SVM/IF rows, with the β-VAE row used as a comparator throughout this paper (e.g., Table 28), are imported from [5] rather than rerun

here. We confirm that the feature-selection pipeline, the 20 selected indices, and the BoT-IoT/IoT-23/ToN-IoT source files are the same artefacts described in Section 3 (this paper's codebase descends directly from [5]'s), so the comparison is not an apples-to-oranges mismatch of unrelated datasets. We cannot, however, independently re-verify from inside this paper's repository that [5]'s own train/test split, random seed, and hyperparameter choices for AE/RF/SVM/IF were identical to the protocol used here (that verification would require access to [5]'s own run logs, which are outside this repository), and Section 3.3's row-duplication finding raises an additional, specific concern: if [5]'s classical-baseline numbers were generated before or after the dataset files were expanded from 6000 to 33,000–34,000 rows, the two studies' BoT-IoT partitions may not be row-for-row identical. We therefore do not describe this comparison as strictly like-for-like, and readers should treat the AE/RF/SVM/IF rows as an informative but not perfectly controlled reference point, distinct from this paper's own newly-generated, cross-checked numbers.

Classical baselines degrade substantially cross-dataset despite strong in-domain performance: RF achieves an AUC of 0.9990 on BoT-IoT but collapses to 0.582 on ToN-IoT, reflecting over-fitting to BoT-IoT-specific feature correlations that do not transfer. All three TCN-family/VAE models retain AUC ≥ 0.947 across every dataset, but—as emphasised throughout this paper—AUC alone does not establish operational reliability; the threshold-granularity diagnosis in Section 4.7 is the clearest demonstration that two models with near-identical, strong AUC (TCN-Pred: 0.9986–0.9996; the same TCN-VAE under either threshold calibration: 0.9338–0.9986) can differ by more than 30× in FPR depending on calibration alone.

Table 20. ROC-AUC comparison of TCN-Pred and TCN-Det against the per-flow β-VAE and classical baselines. Classical results from [5]. **Bold** indicates best result per column.

Model	BoT-IoT	IoT-23	ToN-IoT
TCN-Pred (ours)	**0.9996**	0.9879	0.9471
TCN-Det (ours, ablation)	**0.9996**	**0.9899**	**0.9513**
β-VAE [5]	**0.9996**	0.9882	0.9465
AE	0.9840	0.8110	0.7020
RF	0.9990	0.6730	0.5820
SVM	0.9760	0.7020	0.6110
IF	0.9650	0.6890	0.5930

Bold values indicate the best-performing model for each dataset. **TCN-Det's marginally higher ROC-AUC than TCN-Pred on IoT-23 and ToN-IoT does not translate into better operational (fixed-threshold) performance**: at the fixed $\tau = \mu + 3\sigma$ operating point, TCN-Det's FPR (7.1–37.6%, Table 18) is 5–10× worse than TCN-Pred's (0.65–13%). A reader comparing only this table's AUC column would draw the wrong operational conclusion; ROC-AUC ranks all possible thresholds and says nothing about the one threshold actually deployed.

4.10. Multi-Seed Reliability

To distinguish genuine effects from run-to-run initialisation noise, we retrained TCN-Pred (canonical, excluded-target), the target-included ablation variant, and TCN-Det across 5 seeds (42–46), reporting mean ± standard deviation for ROC-AUC, PR-AUC, F1, and FPR, and a paired bootstrap 95% confidence interval (2000 resamples) for the FPR difference between TCN-Pred and the target-included variant on the same evaluated flows at seed 42 (Table 21). **Cluster-level bootstrap sensitivity check.** Row-level bootstrap resampling assumes independent draws, which Section 3.3 shows does not strictly hold: each provided file contains only ≈6000 genuinely distinct underlying flows, each duplicated 5.5–5.7 times on average. We tested whether this affects the interval above by recomputing DBSCAN duplicate-cluster assignments directly from the raw dataset files, verifying the recovered assignment against each dataset's saved test split, and re-running the same paired bootstrap as a *cluster-level* (block) resample—drawing whole duplicate clusters with replacement instead of individual flows (`src/cluster_bootstrap_sensitivity.py`).

The result: cluster-level interval widths are within 0.1–0.9% of the row-level widths on IoT-23 and ToN-IoT (a width ratio of 0.9989 and 0.9912 respectively; both intervals still exclude a large effect and agree with the row-level conclusion), and both collapse to a degenerate zero-width interval on BoT-IoT because the excluded/included FPR values are identical at this sample size regardless of resampling unit. We attribute the near-identical widths to the small mean cluster size among valid benign evaluation flows in this experiment ($\approx$1.7 rows/cluster after the first-W/attack-flow exclusions), which limits how much block resampling can differ from row resampling here; datasets or statistics with larger surviving clusters could show a bigger effect, so this check should not be read as establishing that row-level and cluster-level bootstraps agree in general (Section 5.6).

Paired bootstrap intervals beyond FPR. In addition to the FPR intervals above, we computed paired flow-level bootstrap 95% intervals (500 resamples) for F1 and PR-AUC differences between target-excluded and target-included predictors. The F1 intervals were $[-0.00045, 0.00076]$, $[-0.00091, 0.00215]$, and $[-0.00167, 0.00183]$ on BoT-IoT, IoT-23, and ToN-IoT; the corresponding PR-AUC intervals were $[-0.000016, 0.000026]$, $[-0.000153, 0.000062]$, and $[-0.000390, 0.000144]$. Every interval includes zero, reinforcing that target exposure does not produce a statistically distinguishable change under this controlled comparison.

Table 21. Multi-seed reliability (5 seeds, 42–46): mean $\pm$ standard deviation.

Model	Dataset	ROC-AUC	PR-AUC	F1	FPR
TCN-Pred (excluded, canonical)	BoT-IoT	0.9996 $\pm$ 0.0000	0.9996 $\pm$ 0.0000	0.9911 $\pm$ 0.0004	0.0062 $\pm$ 0.0004
	IoT-23	0.9878 $\pm$ 0.0003	0.9873 $\pm$ 0.0003	0.9406 $\pm$ 0.0011	0.0561 $\pm$ 0.0007
	ToN-IoT	0.9468 $\pm$ 0.0006	0.9481 $\pm$ 0.0006	0.8751 $\pm$ 0.0014	0.1349 $\pm$ 0.0019
Target-included variant	BoT-IoT	0.9996 $\pm$ 0.0000	0.9996 $\pm$ 0.0000	0.9910 $\pm$ 0.0004	0.0064 $\pm$ 0.0001
	IoT-23	0.9879 $\pm$ 0.0003	0.9874 $\pm$ 0.0003	0.9406 $\pm$ 0.0013	0.0563 $\pm$ 0.0014
	ToN-IoT	0.9469 $\pm$ 0.0005	0.9481 $\pm$ 0.0004	0.8755 $\pm$ 0.0012	0.1347 $\pm$ 0.0015
TCN-Det	BoT-IoT	0.9996 $\pm$ 0.0001	0.9995 $\pm$ 0.0002	0.9698 $\pm$ 0.0049	0.0622 $\pm$ 0.0105
	IoT-23	0.9889 $\pm$ 0.0008	0.9881 $\pm$ 0.0009	0.9083 $\pm$ 0.0134	0.1958 $\pm$ 0.0359
	ToN-IoT	0.9534 $\pm$ 0.0026	0.9534 $\pm$ 0.0028	0.8394 $\pm$ 0.0117	0.3574 $\pm$ 0.0432

TCN-Pred and the target-included variant are statistically indistinguishable on every metric across 5 seeds (std $\leq$ 0.0019 FPR for both), reinforcing Table 11's single-seed result. Paired bootstrap 95% CIs (2000 resamples, seed 42) for FPR(TCN-Pred)$-$FPR(target-included) include zero on all three datasets: BoT-IoT $[-0.0009, 0.0000]$, IoT-23 $[-0.0025, 0.0012]$, ToN-IoT $[-0.0028, 0.0027]$—the difference is not statistically distinguishable from zero. TCN-Det shows both a higher mean FPR and a much larger seed-to-seed standard deviation (up to 0.0432 on ToN-IoT, roughly 20–25$\times$ TCN-Pred's) than either variational model, indicating the deterministic model's reported single-seed FPR (Table 18) is, itself, considerably less reliable.

Remaining multi-seed checks. We also retrained the per-flow β-VAE and repeated TCN-Pred/TCN-Det with a sample-disjoint benign calibration block over the same five seeds. Table 22 ensures that the β-VAE is represented by multiple runs and that TCN-Det is evaluated under sample-disjoint rather than only same-sample threshold calibration.

Table 22. Additional 5-seed checks (seeds 42–46): mean $\pm$ standard deviation. The β-VAE uses its canonical per-flow source threshold; TCN models use an 80/20 sample-disjoint benign fit/calibration split with no crossing windows.

Model	Dataset	ROC-AUC	PR-AUC	F1	FPR
Per-flow β-VAE	BoT-IoT	0.9996 $\pm$ 0.0000	0.9996 $\pm$ 0.0000	0.9914 $\pm$ 0.0008	0.0059 $\pm$ 0.0003
	IoT-23	0.9880 $\pm$ 0.0005	0.9875 $\pm$ 0.0005	0.9411 $\pm$ 0.0021	0.0566 $\pm$ 0.0016
	ToN-IoT	0.9465 $\pm$ 0.0016	0.9475 $\pm$ 0.0016	0.8748 $\pm$ 0.0023	0.1364 $\pm$ 0.0040
TCN-Pred, separate calibration	BoT-IoT	0.9996 $\pm$ 0.0000	0.9996 $\pm$ 0.0000	0.9916 $\pm$ 0.0005	0.0073 $\pm$ 0.0004
	IoT-23	0.9878 $\pm$ 0.0005	0.9872 $\pm$ 0.0005	0.9420 $\pm$ 0.0019	0.0595 $\pm$ 0.0020
	ToN-IoT	0.9467 $\pm$ 0.0015	0.9478 $\pm$ 0.0014	0.8751 $\pm$ 0.0026	0.1432 $\pm$ 0.0032

Table 22. *Cont.*

Model	Dataset	ROC-AUC	PR-AUC	F1	FPR
TCN-Det, separate calibration	BoT-IoT	0.9994 ± 0.0003	0.9994 ± 0.0003	0.9827 ± 0.0093	0.0339 ± 0.0202
	IoT-23	0.9887 ± 0.0021	0.9883 ± 0.0022	0.9276 ± 0.0189	0.1403 ± 0.0523
	ToN-IoT	0.9518 ± 0.0039	0.9514 ± 0.0045	0.8606 ± 0.0202	0.2700 ± 0.0778

The contiguous calibration block is sample-disjoint but is not claimed to be chronological because timestamps and capture identifiers are absent. The β-VAE and TCN-Pred distributions overlap closely, confirming parity rather than a temporal-model improvement. Separate calibration improves TCN-Det but does not remove its higher and much less stable FPR.

4.11. Sample-Disjoint Calibration Partition and Normality Diagnostics

The canonical threshold is calibrated on the same benign flows used to fit the model's weights, which can be optimistic. We retrained TCN-Pred using only the first 80% of the benign BoT-IoT training sequence (10,880 flows, contiguous block) to fit weights, holding out the remaining 20% (2720 flows, 10,860/2700 evaluable (context, target) pairs after the window exclusion) as a calibration block that never contributes a gradient step; context windows are not allowed to span the fit/calibration boundary. As discussed in Section 3.2, this is a sample-disjoint, contiguous-block separation, not a verified temporal split.

Normality diagnostics. On the calibration-block benign errors: skewness $= 0.656$, excess kurtosis $= 0.604$ (both indicate a mildly right-skewed, heavier-than-Gaussian-tailed distribution, as expected for a non-negative MSE statistic); Shapiro–Wilk $W = 0.9754$, $p = 3.5 \times 10^{-21}$ (formally rejects exact normality at any conventional significance level, unsurprising given $n = 2700$); Q–Q correlation $r = 0.9877$ (the bulk of the distribution is close to Gaussian, with deviation concentrated in the tails, which is exactly where a $\mu + 3\sigma$ rule is most sensitive to misspecification). Table 23 compares the resulting same-sample and sample-disjoint thresholds under both a Gaussian and an empirical rule.

Table 23. Threshold comparison: same-sample calibration vs. sample-disjoint calibration-block calibration, Gaussian ($\mu + 3\sigma$) vs. empirical 99.7th-percentile rule.

Dataset	Threshold Rule	F1	FPR	Recall
BoT-IoT	Same-sample (original)	0.9922	0.0059	0.9903
	Separate-cal., Gaussian	0.9917	0.0068	0.9903
	Separate-cal., empirical 99.7th pct	0.9895	0.0041	0.9832
IoT-23	Same-sample (original)	0.9426	0.0584	0.9435
	Separate-cal., Gaussian	0.9439	0.0602	0.9474
	Separate-cal., empirical 99.7th pct	0.9382	0.0435	0.9219
ToN-IoT	Same-sample (original)	0.8775	0.1390	0.8903
	Separate-cal., Gaussian	0.8774	0.1448	0.8946
	Separate-cal., empirical 99.7th pct	0.8747	0.1089	0.8618

Same-sample and separate-calibration-block thresholds agree closely (FPR differences ≤ 0.006), indicating the original same-sample threshold was not materially optimistic for TCN-Pred. The empirical 99.7th-percentile rule consistently trades recall for a lower FPR relative to the Gaussian rule, consistent with the mild right-skew documented above.

4.12. Split-Conformal Threshold Baseline

Section 2.4 cites conformal prediction [23,24] as a distribution-free alternative to a Gaussian $\mu + 3\sigma$ rule. We implement a split-conformal threshold using the same sample-disjoint clean calibration block as Section 4.11 ($n = 2700$ benign errors, 0% attack): for a target miscoverage level α, $\tau_{\text{conformal}}$ is the $\lceil (n+1)(1-\alpha) \rceil$-th smallest calibration error, which guarantees $P(\text{score}_{\text{new}} > \tau_{\text{conformal}}) \leq \alpha$ for a new sample *exchangeable with the calibration set*. We test $\alpha \in \{0.05, 0.01, 0.0027\}$ (the last approximating the two-sided Gaussian 3-sigma rule's nominal rate) and report the achieved FPR on all three datasets in Table 24.

Table 24. Split-conformal threshold: nominal miscoverage bound α vs. achieved FPR. Calibration set: sample-disjoint clean BoT-IoT benign block ($n = 2700$).

Dataset	Nominal α	τ	Achieved FPR	F1
BoT-IoT (in-domain)	0.05	1.0441	0.0448	0.9778
	0.01	1.2588	0.0091	0.9920
	0.0027	1.3604	0.0035	0.9890
IoT-23 (cross-dataset)	0.05	1.0441	0.1864	0.9120
	0.01	1.2588	0.0657	0.9443
	0.0027	1.3604	0.0392	0.9363
ToN-IoT (cross-dataset)	0.05	1.0441	0.3513	0.8381
	0.01	1.2588	0.1582	0.8768
	0.0027	1.3604	0.1037	0.8702

On BoT-IoT—the same distribution as the calibration set—achieved an FPR that tracks the nominal bound closely (e.g., 0.0035 achieved vs. 0.0027 nominal). On IoT-23 and ToN-IoT, the achieved FPR exceeds the nominal bound by roughly one to two orders of magnitude (e.g., $\alpha = 0.0027$ gives 3.9% achieved FPR on IoT-23 and 10.4% on ToN-IoT). This is expected: split conformal's marginal coverage guarantee requires the calibration and deployment samples to be exchangeable, which does not hold under the cross-dataset domain shift studied in this paper. This failure mode is precisely why Gibbs and Candès [24] propose *adaptive* conformal inference for distribution shift; a full adaptive-conformal implementation is left to future work (Section 5.6).

4.13. Adaptive Thresholding with Clean Calibration

We compare the fixed threshold against Robust MAD and EMA adaptation using the clean (0% attack), sample-disjoint calibration block from Section 4.11, and sweep the EMA adaptation rate $\alpha \in \{0.001, 0.01, 0.05\}$ (Table 25).

Table 25. Adaptive thresholding with clean calibration (0% attack contamination in the calibration block). Robust MAD tau (clean cal.) $= 1.0387$; fixed/EMA start from $\tau = 1.2781$, $\mu = 0.6418$, $\sigma = 0.2121$.

Dataset	Strategy	F1	FPR
BoT-IoT	Fixed ($\mu + 3\sigma$, clean cal.)	**0.9920**	0.0077
	Robust MAD (clean cal.)	0.9771	0.0463
	EMA, $\alpha = 0.001$	0.9869	0.0251
	EMA, $\alpha = 0.01$	0.9790	0.0422
	EMA, $\alpha = 0.05$	0.9748	0.0507
IoT-23	Fixed ($\mu + 3\sigma$, clean cal.)	**0.9411**	0.0623
	Robust MAD (clean cal.)	0.9107	0.1898
	EMA, $\alpha = 0.001$	0.8612	0.0182
	EMA, $\alpha = 0.01$	0.1783	0.0027
	EMA, $\alpha = 0.05$	0.0799	0.0003
ToN-IoT	Fixed ($\mu + 3\sigma$, clean cal.)	**0.8771**	0.1463
	Robust MAD (clean cal.)	0.8363	0.3574
	EMA, $\alpha = 0.001$	0.4884	0.0192
	EMA, $\alpha = 0.01$	0.1281	0.0030
	EMA, $\alpha = 0.05$	0.1257	0.0006

Even with a clean, uncontaminated calibration block, Robust MAD still gives a higher FPR than the fixed rule on every dataset: for this mildly right-skewed error distribution, median + 3·MAD sits lower than mean + 3σ, so it is the *stricter* (more permissive-to-flag) rule here, not the reverse. EMA is highly sensitive to α: at $\alpha = 0.001$, it approximates the fixed threshold reasonably on BoT-IoT but under-performs on the more distribution-shifted IoT-23/ToN-IoT, while a larger α causes the threshold to drift upward with elevated target-domain "likely-benign" errors, collapsing recall. These findings now rest on a fair, uncontaminated comparison and support restricting the "fixed threshold is preferable" conclusion to the specific alternatives and settings tested here, not as a general claim against adaptive thresholding.

4.14. Realistic Attack-Prevalence Reconstruction

We reconstruct the evaluation stream at each target prevalence by inserting contiguous attack episodes (run length $\sim$Uniform$[3, 15]$, drawn from the dataset's own attack flows) into an all-benign backbone, and we recompute contexts and predictions from scratch with the trained model and a fixed threshold, unchanged. Table 26 reports the mean over 5 random layouts per prevalence level. We describe this explicitly as a *controlled prevalence reconstruction using the available row ordering*, not a natural or verified temporal

stream: Section 3.2 already establishes that we cannot confirm that the provided row order reflects original capture chronology, and inserting contiguous attack episodes into a benign backbone does not restore that chronology even where it recreates locally plausible context composition. The point of this experiment is to vary context attack density under a controlled, reproducible protocol, not to claim genuine temporal validity for the resulting sequence. Accordingly, we present it throughout as a controlled sensitivity analysis of context composition and attack prevalence, not as independent real-world or natural-prevalence validation.

Table 26. Realistic attack-prevalence reconstruction: contiguous attack episodes inserted into a benign backbone; contexts and predictions recomputed from scratch at each prevalence level (mean ± std over 5 random layouts).

Dataset	Prevalence	ROC-AUC	F1	FPR
BoT-IoT	1%	0.9996	0.756	0.0061 ± 0.0001
	2%	0.9997	0.865	0.0062 ± 0.0001
	5%	0.9996	0.939	0.0064 ± 0.0002
	10%	0.9996	0.969	0.0060 ± 0.0001
	20%	0.9996	0.983	0.0062 ± 0.0002
	50%	0.9996	0.991	0.0063 ± 0.0003
IoT-23	1%	0.9901	0.251	0.0555 ± 0.0006
	2%	0.9881	0.402	0.0564 ± 0.0007
	5%	0.9878	0.625	0.0563 ± 0.0006
	10%	0.9891	0.773	0.0553 ± 0.0006
	20%	0.9883	0.867	0.0565 ± 0.0006
	50%	0.9877	0.940	0.0553 ± 0.0011
ToN-IoT	1%	0.9479	0.116	0.1322 ± 0.0008
	2%	0.9486	0.214	0.1326 ± 0.0014
	5%	0.9484	0.403	0.1330 ± 0.0004
	10%	0.9467	0.574	0.1331 ± 0.0011
	20%	0.9463	0.734	0.1318 ± 0.0009
	50%	0.9464	0.876	0.1324 ± 0.0012

Under this properly reconstructed protocol—contiguous episodes, contexts recomputed from scratch, and no score reweighting—FPR remains flat within measurement noise across the full 1–50% prevalence range on all three datasets (BoT-IoT: 0.0060–0.0064; IoT-23: 0.0553–0.0565; ToN-IoT: 0.1318–0.1332). Recall stays above 0.88 throughout. This confirms, on stronger methodological footing, the FPR-stability claim of the original (reweighting-based) experiment.

Figure 8 plots this same reconstructed FPR/F1/recall trend against attack prevalence.

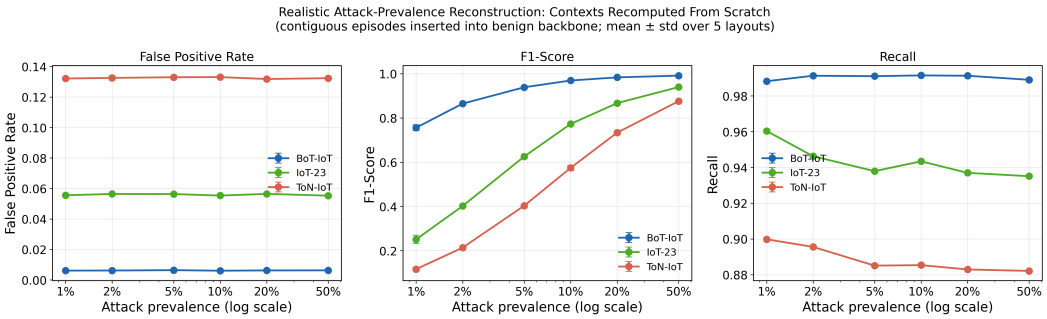


Figure 8. FPR , F1, and recall vs. attack prevalence under the reconstructed protocol (contiguous episodes inserted into a benign backbone, and contexts and predictions recomputed from scratch; mean ± std over 5 random layouts per prevalence level). This replaces the dense red-shaded temporal trace used for the original, score-reweighting version of this experiment (Figure 7), which was difficult to interpret; FPR is visibly flat across the full prevalence range on all three datasets, while F1 rises with prevalence, as explained in the text.

4.15. Feature Correlation and Redundancy Analysis

The top-ranked ANOVA–MI features (Table A1) are dominated by packet-length and packet-volume statistics that could plausibly be highly collinear, and a univariate filter

such as ANOVA–MI cannot see or penalise this redundancy directly. We computed the Spearman rank correlation matrix for all $\binom{20}{2} = 190$ pairs of the 20 selected features, fitted on the BoT-IoT training partition only (27,200 rows; Figure 9).

Contrary to that plausible prior, the measured redundancy is modest: median $|\rho| = 0.018$, maximum $|\rho| = 0.655$, and *zero* of the 190 pairs reach $|\rho| \geq 0.80$ or $|\rho| \geq 0.90$. The most correlated pair is `bwd_pkt_len_min` and `flow_duration` ($\rho = 0.655$), followed by several other packet-length/volume pairs in the 0.64–0.65 range (full list in `results/feature_correlation_analysis.json`). We report this as a genuine, if unexpected, negative result rather than dismiss the underlying redundancy concern: the top-12 features being thematically similar packet-length/volume statistics (Appendix A) does not, in this dataset, translate into strong pairwise rank redundancy. This does not establish that the features are non-redundant in a higher-order, multivariate sense (correlation is pairwise and linear-rank only, and cannot detect joint or non-monotonic redundancy), which is exactly why Section 4.17 benchmarks a redundancy-aware multivariate selector (mRMR) directly rather than relying on this diagnostic alone.

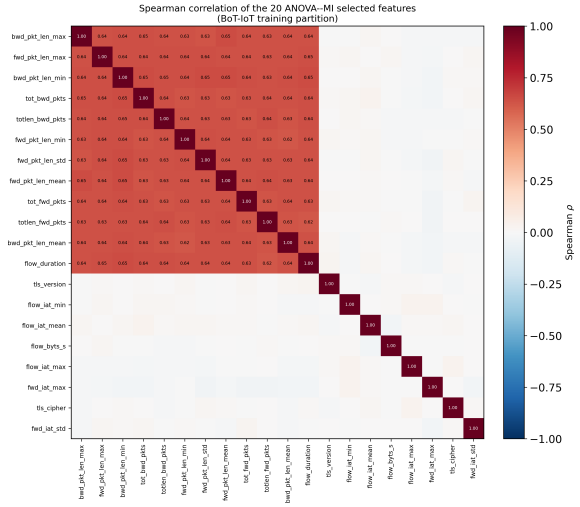


Figure 9. Spearman correlation matrix of the 20 ANOVA–MI-selected features, BoT-IoT training partition. No pair reaches $|\rho| \geq 0.80$; the strongest pairwise correlations ($|\rho| \approx 0.65$) occur among packet-length and packet-volume statistics.

4.16. ANOVA Distributional Assumptions and a Non-Parametric Ranking Comparison

Section 4.11 tests normality of *anomaly errors* at evaluation time; that is a different question from whether the ANOVA F-test's assumptions hold for the *candidate features themselves* at the feature-selection stage, which is a distinct question worth checking directly. For each of the 30 candidate features, we computed per-class (benign vs. attack) skewness, excess kurtosis, and a Levene test for equality of variance across classes on the BoT-IoT training partition, with a Benjamini–Hochberg false-discovery-rate correction applied across the 30 Levene p-values (a single test repeated 30 times without correction would overstate how many features are genuinely heteroscedastic).

Unexpectedly, none of the 30 features shows strong skew ($|\text{skew}| > 1$) or heavy tails ($|\text{excess kurtosis}| > 3$) in either class (0/30 for both diagnostics); a total of 8/30 (27%) are flagged as heteroscedastic at a BH-corrected $q < 0.05$. This is a substantially milder violation of ANOVA's assumptions than a prior expectation of heavy-tailed, right-skewed flow statistics would predict, and it is, itself, informative: genuine network flow-duration and byte-count statistics are essentially never this close to Gaussian per class, which is consistent with—and adds further, independent evidence for—the artefact-provenance concerns raised in Section 3.2 and quantified directly in Section 3.3. We report the measured

diagnostic rather than the expected one and do not use it to claim that the ANOVA assumptions are fully satisfied; heteroscedasticity is still present in over a quarter of features, and the Kruskal–Wallis test, a rank-based test that does not require normality and remains valid under heteroscedasticity as a stochastic-dominance comparison, is a more defensible ranking criterion regardless.

We therefore also ranked all 30 features by the Kruskal–Wallis H statistic (benign vs. attack) and compared its top-20 set against the canonical ANOVA–MI top 20. The two rankings agree on 16 out of 20 features (80% overlap); ANOVA–MI selects `flow_iat_min`, `flow_byts_s`, `tls_cipher`, and `fwd_iat_std`, whereas Kruskal–Wallis, instead, selects four different rate/inter-arrival-timing features. Both rankings agree that `tls_version` belongs in the top 20, but only ANOVA–MI keeps `tls_cipher`—i.e., the weaker of the two TLS-metadata features is exactly the one boundary case that a distribution-free ranking does not reproduce. Given this is a ranking-robustness check rather than a claim of assumption satisfaction, we read the 80% agreement as reassuring rather than decisive: the ANOVA–MI selection is not an artefact of a badly violated normality assumption, but it is also not perfectly reproduced by a parametric-assumption-free alternative. Section 4.17 includes a Kruskal–Wallis-only feature set (`kw_only20`) in the full multivariate benchmark below.

4.17. Multivariate, Redundancy-Aware Feature Selection (mRMR)

ANOVA–MI is univariate: it scores each feature independently against the label and cannot penalise redundancy between jointly-informative features. We implemented minimum-Redundancy-Maximum-Relevance (mRMR, MID variant; [33]) as a multivariate, redundancy-aware benchmark. No third-party mRMR implementation was available in this environment, so the standard criterion was implemented directly (`src/mrmr_selection.py`) and documented here: relevance is the mutual information between a candidate feature and the label, $I(f;y)$; redundancy is the mean mutual information between a candidate and the already selected set, $\frac{1}{|S|}\sum_{s\in S} I(f;s)$; features are added greedily to maximise $I(f;y) - \frac{1}{|S|}\sum_{s\in S} I(f;s)$. Both terms use the scikit-learn KNN-based mutual-information estimator (`random_state=42`), fitted on BoT-IoT training data only; mRMR is, like ANOVA–MI, a **supervised** selector, since relevance requires the attack labels.

TCN-Pred was retrained identically (the same harness, hyperparameters, and seed protocol as every other controlled experiment in this paper) on seven feature sets: all 30 features (no selection), ANOVA-only-20, MI-only-20, Kruskal–Wallis-only-20 (Section 4.16), and mRMR at $k \in \{10, 15, 20\}$. mRMR-20, as the principal redundancy-aware comparator to the canonical ANOVA–MI selection, was run at 5 seeds (42–46); every other feature set was run at seed 42 only, reported as a single run rather than mixed into the same column as a multi-seed mean without distinction. Table 27 reports the resulting metrics for all seven feature sets.

4.18. Feature-Selection Stability Under Resampling

Both ANOVA–MI and mRMR are fitted on a single BoT-IoT training split, so it is fair to ask whether the selected set would look materially different under resampling. For ANOVA–MI, we refit the full selector on 100 class-stratified bootstrap resamples of the BoT-IoT training partition (full size, $n = 27{,}200$, with replacement). mRMR's per-resample cost is dominated by its 30×30 pairwise mutual-information matrix, which made 100 full-size refits computationally excessive in the available time; we therefore used 50 class-stratified bootstrap resamples at a reduced size ($n = 2000$), a lighter protocol reported explicitly as such rather than presented as equivalent to the ANOVA–MI one. Both selectors are refit independently on each resample (`src/feature_selection_stability.py`).

ANOVA–MI is moderately stable: the mean pairwise Jaccard similarity across the 100 resamples is 0.718 ± 0.077, and 14 of 30 candidate features (the 12 packet-length/volume features at indices 0–11, plus `flow_iat_min` and `tls_version`) are selected in $\geq$80% of resamples. The remaining boundary features—including, notably, `tls_cipher`—are selected in 20–80% of resamples, i.e., they are genuinely unstable boundary cases rather than reliably in or reliably out. This is the same feature (`tls_cipher`) that Section 4.16 finds to not survive the switch to Kruskal–Wallis ranking; two independent diagnostics agree that the weaker of the two TLS-metadata features is the least robust part of the canonical selection, while `tls_version` is comparatively robust on both.

mRMR is somewhat less stable under its (lighter) resampling protocol: mean pairwise Jaccard 0.641 ± 0.072, with only the same 12 packet-length/volume features reaching the 80% threshold; neither TLS feature, nor any of the timing/rate features mRMR sometimes selects in place of ANOVA–MI's choices (Section 4.17), is robustly stable under resampling for mRMR. We read this as a genuine, if modest, cost of redundancy-aware selection in this feature space: avoiding redundant packet-length/volume features pushes mRMR toward a larger pool of comparably informative, weakly discriminative alternatives among the timing/rate and TLS features, and which one of them gets picked is more resample-sensitive than ANOVA–MI's more concentrated ranking. Full per-feature selection frequencies for both selectors are in `results/feature_selection_stability.json`.

Table 27. Feature-selection benchmark: TCN-Pred retrained identically on each feature set. mRMR-20 reports mean $\pm$ SD over 5 seeds (42–46); all other rows are single-seed (seed 42). **Bold** marks the canonical ANOVA–MI-20 row (Table 6) for reference, not recomputed here.

Dataset	Feature Set (k)	ROC-AUC	F1	Precision/Recall	FPR
BoT-IoT	All 30 (no selection)	0.9957	0.9273	0.9919/0.8705	0.0071
	ANOVA-only (20)	0.9996	0.9902	0.9943/0.9861	0.0056
	MI-only (20)	0.9996	0.9904	0.9935/0.9873	0.0065
	Kruskal–Wallis-only (20)	0.9997	0.9899	0.9938/0.9861	0.0062
	mRMR (10)	1.0000	0.9959	0.9918/1.0000	0.0083
	mRMR (15)	1.0000	0.9930	0.9860/1.0000	0.0142
	mRMR (20), 5 seeds	0.9993 ± 0.0000	0.9874 ± 0.0009	0.9875/0.9873	0.0125 ± 0.0003
	ANOVA–MI (20, canonical)	**0.9996**	**0.9913**	**0.9935/0.9891**	**0.0065**
IoT-23	All 30 (no selection)	0.9694	0.8515	0.9705/0.7585	0.0231
	ANOVA-only (20)	0.9870	0.9437	0.9454/0.9420	0.0544
	MI-only (20)	0.9878	0.9400	0.9442/0.9359	0.0554
	Kruskal–Wallis-only (20)	0.9867	0.9384	0.9421/0.9347	0.0575
	mRMR (10)	0.9886	0.8964	0.8132/0.9985	0.2296
	mRMR (15)	0.9929	0.9468	0.9059/0.9915	0.1031
	mRMR (20), 5 seeds	0.9873 ± 0.0003	0.9443 ± 0.0016	0.9458/0.9427	0.0541 ± 0.0015
	ANOVA–MI (20, canonical)	**0.9879**	**0.9406**	**0.9437/0.9365**	**0.0560**
ToN-IoT	All 30 (no selection)	0.9142	0.7643	0.9105/0.6586	0.0648
	ANOVA-only (20)	0.9415	0.8690	0.8579/0.8803	0.1460
	MI-only (20)	0.9471	0.8759	0.8697/0.8821	0.1323
	Kruskal–Wallis-only (20)	0.9444	0.8706	0.8666/0.8745	0.1347
	mRMR (10)	0.9370	0.8036	0.6766/0.9891	0.4732
	mRMR (15)	0.9603	0.8726	0.7918/0.9717	0.2558
	mRMR (20), 5 seeds	0.9419 ± 0.0006	0.8669 ± 0.0019	0.8565/0.8776	0.1472 ± 0.0016
	ANOVA–MI (20, canonical)	**0.9471**	**0.8788**	**0.8722/0.8855**	**0.1299**

We test whether a *smaller* multivariate-selected set matches ANOVA–MI's performance. It does not: mRMR at $k = 10$ and $k = 15$ achieves near-perfect BoT-IoT recall ($\geq$0.99) but at a severe cross-dataset FPR cost (IoT-23: up to 22.96% at $k = 10$; ToN-IoT: up to 47.32% at $k = 10$)—roughly 2–9$\times$ worse than the canonical selection, indicating these smaller redundancy-optimised sets overfit the source-domain threshold and generalise poorly out-of-domain. Only at $k = 20$ does mRMR approach ANOVA–MI's cross-dataset FPR (IoT-23: 0.0541 vs. 0.0560; ToN-IoT: 0.1472 vs. 0.1299—mRMR-20 is slightly *worse* on ToN-IoT, slightly *better* on IoT-23), with essentially indistinguishable ROC-AUC/F1. ANOVA-only, MI-only, and Kruskal–Wallis-only are all close to each other and to the canonical fusion, consistent with Section 4.16's 80% ranking overlap. The all-30 (no-selection) row, again, shows the lowest FPR at the largest recall cost, reproducing Table 28's finding under a different comparison set. Overall: the underlying redundancy concern is real (Section 4.15 finds it milder than expected, but mRMR does select a different, less redundant set), but addressing it with a smaller feature budget costs cross-dataset reliability rather than improving it; a redundancy-aware selector only matches the univariate fusion's operating point once given the same budget ($k = 20$), at which point it is a lateral, not a superior, alternative.

4.19. TLS/Encrypted-Flow Coverage

Given the "encrypted IoT" framing of this study, we report what proportion of flows in each partition actually carry a TLS handshake. Before computing any such proportion, we first checked how a missing/default TLS value would be encoded in the four TLS-metadata columns (indices 26–29: `tls_version`, `tls_cipher`, and the two columns Table A1 associates with JA3/JA3S) because the earlier pipeline documentation described these columns as frequency-encoded categorical values with unseen values mapped to zero.

Table 28. Label-free feature selection and a no-selection baseline vs. labelled ANOVA–MI. TCN-Pred retrained identically on each feature space.

Dataset	Selection Method	ROC-AUC	F1	FPR
BoT-IoT	All 30 features (no selection)	0.9957	0.9276	0.0074
	Benign-only variance (top-20, unsupervised)	0.6982	0.0815	0.0065
	PCA(20) (unsupervised)	0.7695	0.1319	0.0065
	ANOVA–MI (labelled, canonical)	**0.9996**	**0.9913**	**0.0065**
IoT-23	All 30 features (no selection)	0.9692	0.8493	**0.0231**
	Benign-only variance (top-20, unsupervised)	0.5921	0.0526	0.0116
	PCA(20) (unsupervised)	0.6449	0.0659	0.0082
	ANOVA–MI (labelled, canonical)	**0.9879**	**0.9406**	0.0544
ToN-IoT	All 30 features (no selection)	0.9143	0.7642	**0.0645**
	Benign-only variance (top-20, unsupervised)	0.6006	0.0611	0.0128
	PCA(20) (unsupervised, 83.3% var. explained)	0.5787	0.0554	0.0137
	ANOVA–MI (labelled, canonical)	**0.9471**	**0.8779**	0.1293

The two *unsupervised dimensionality-reduction* baselines (variance, PCA—both compress 30 features down to 20) collapse to near-chance ranking (AUC 0.58–0.77) and very low recall (F1 0.05–0.13, driven by high precision but almost no attack flows crossed). The *all-features, no-selection* baseline tells a more nuanced story: it retains strong ranking (AUC 0.914–0.996) and, on the two cross-dataset targets, actually achieves a *lower* FPR than the labelled ANOVA–MI selection (IoT-23: 0.023 vs. 0.054; ToN-IoT: 0.065 vs. 0.129), but at a substantial recall/F1 cost (F1 0.764–0.928 vs. 0.878–0.991). So the labelled ANOVA–MI stage is not doing the work of rescuing an otherwise uninformative feature space—the full 30-feature space is already informative; it is trading a modest amount of FPR for a large gain in recall, and it is specifically *unsupervised dimensionality reduction to 20 features*, not feature selection or reduction *per se*, that destroys the signal seen in the variance/PCA rows above.

We found that this encoding cannot be verified or exploited in the artefacts available to us. In every partition (BoT-IoT train/test, IoT-23, ToN-IoT) and at both the "raw"-CSV stage (`data/raw/*.csv`) and the fully processed stage, all four TLS columns are continuous, approximately-$\mathcal{N}(0,1)$-ranged floating-point values with the number of distinct values equal or almost equal to the number of rows, zero exact-zero entries, and no value repeated more than a handful of times; this is inconsistent with a frequency-encoded categorical column (which would show many exact ties at each category's frequency) or an explicit default/unknown placeholder (which would show a spike at one value). No categorical or placeholder structure was detected in any of the 16 (4 partitions $\times$ 4 columns) checks (full detail in `results/tls_coverage_analysis.json`). This means the columns were already scaled upstream of this study's codebase and the shared artefacts before `data/raw/*.csv` was written; no script in this repository performs that transformation, so it cannot be undone or verified against the description in Section 3.

We do not estimate or invent a TLS-observed proportion for any partition. Instead, we soften the paper's encrypted-traffic framing to the claim that is directly measurable: TLS-metadata removal does not degrade TCN-Pred's ranking performance (Section 4.21), indicating that the results primarily reflect payload-independent flow statistics rather than a TLS-specific detection mechanism, not that a known, high proportion of the evaluated flows are confirmed TLS-encrypted.

4.20. Label-Free Feature Selection

The ANOVA–MI feature-selection stage uses BoT-IoT attack labels, so the overall pipeline is *supervised source-domain feature selection followed by benign-only detector training*,

not a fully unsupervised system, even though the detector itself is trained benign-only. Table 28 compares two label-free alternatives, fitted on BoT-IoT benign training data only, with no class information used anywhere, and a no-selection baseline using all 30 processed features, against the labelled ANOVA–MI selection used throughout the rest of this paper.

4.21. TLS-Metadata-Removed Evaluation

Of the 20 ANOVA–MI-selected features, two are TLS metadata (`tls_version`, `tls_cipher`; Table A1). Table 29 shows the retrainings of TCN-Pred on the remaining 18 features.

Table 29. TCN-Pred performance with TLS metadata features removed (18 features vs. the canonical 20).

Dataset	ROC-AUC	PR-AUC	F1	Prec.	Recall	FPR
BoT-IoT	0.9997	0.9997	0.9937	0.9912	0.9962	0.0088
IoT-23	0.9906	0.9902	0.9494	0.9380	0.9611	0.0636
ToN-IoT	0.9516	0.9518	0.8785	0.8451	0.9146	0.1679

Removing TLS metadata leaves ROC-AUC/F1 unchanged or marginally improved on all three datasets (e.g., ToN-IoT AUC 0.9471→0.9516) at the cost of a somewhat higher FPR on ToN-IoT (0.1293→0.1679). TCN-Pred does not depend on TLS metadata for its ranking performance; the flow-timing/volume statistics alone carry essentially all of the discriminative signal in this feature set.

4.22. Latent Space Analysis

Figure 10 shows t-SNE projections of the TCN-Pred latent means for 800 randomly sampled benign and attack flows per dataset. Given Section 4.5's finding that TCN-Pred's output is close to context-invariant, we now read this figure primarily as a visualisation of how the *target label* separates in a learned feature space, rather than as evidence that the context encoding itself carries rich temporal information.

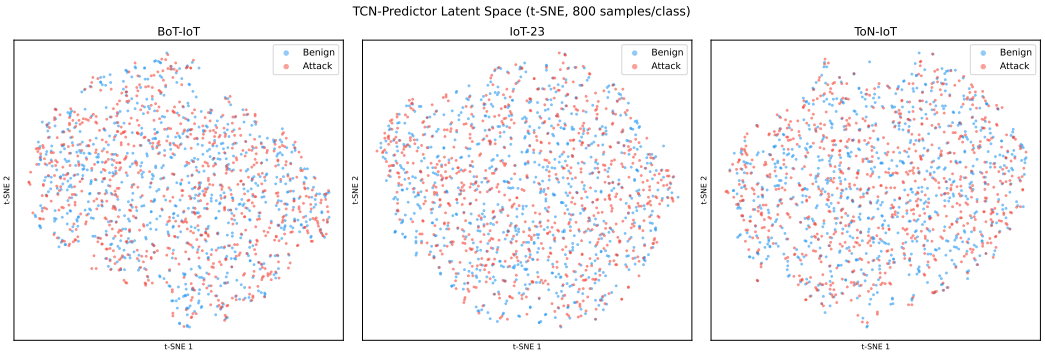


Figure 10. t-SNE projection of TCN-Pred context-window latent means (μ) for 800 randomly sampled evaluation windows per class per dataset. Each point represents the encoding of a W-flow context window; its colour indicates the label of the *immediately following target flow* (blue = benign target; red = attack target). Separation in latent space, therefore, reflects how well the context encoding predicts target-flow class, not the class of the context itself. As a non-linear, stochastic projection, this figure is illustrative rather than confirmatory, and, in light of Section 4.5, it should not be read as evidence of temporal modelling specifically.

Cluster separation is strongest on BoT-IoT and weakens progressively on IoT-23 and ToN-IoT, tracking the progression of ROC-AUC across the three datasets. Because z is close to context-invariant (Section 4.5) and $D_{\mathrm{KL}} \approx 10^{-5}$ (Section 4.24), the more likely explanation for this separation is that z still encodes enough of the *marginal* feature distribution to correlate with the target's class, not that the model has learned a context-conditioned temporal representation.

4.23. Window-Size Sensitivity

Table 30 and Figure 11 report results for $W \in \{5, 10, 20, 50\}$ with $\beta = 2$ fixed.

Table 30. Window-size ablation ($\beta = 2$ fixed). Maximum AUC variation across all W values is 0.0017; maximum FPR variation is 0.009. **Bold** indicates best result per column.

	BoT-IoT		IoT-23		ToN-IoT	
W	**AUC**	**FPR**	**AUC**	**FPR**	**AUC**	**FPR**
5	0.9996	**0.0059**	0.9879	0.0561	0.9468	0.1317
10	**0.9997**	0.0062	**0.9884**	0.0586	0.9474	0.1391
20	0.9996	0.0065	0.9879	0.0560	0.9471	**0.1299**
50	0.9996	0.0068	0.9881	**0.0552**	**0.9485**	0.1309

Bold values indicate the best result for each metric column.

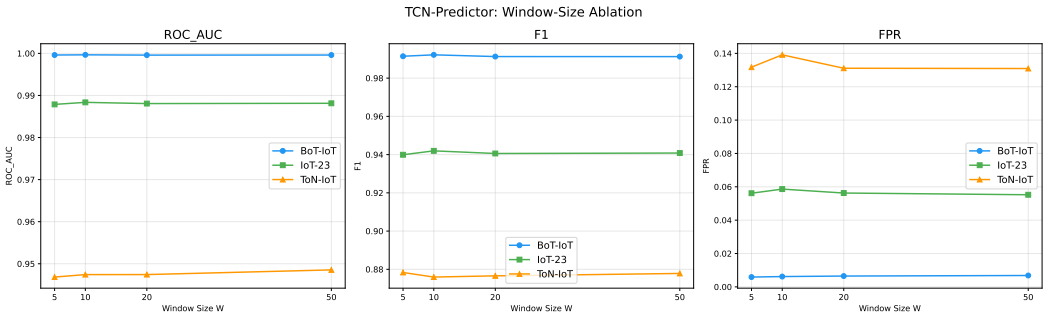


Figure 11. Window -size ablation: ROC-AUC, F1-score, and FPR vs. $W \in \{5, 10, 20, 50\}$ on all three datasets ($\beta = 2$ fixed). Each metric traces a near-horizontal line; maximum AUC variation is 0.0017, and maximum FPR variation is 0.009.

We initially read this flatness as evidence that "the useful regularity in encrypted IoT flows is extremely short-range." In light of Section 4.5—where zeroing the context entirely leaves FPR and F1 within 0.002 of the real context—we now consider a simpler explanation at least as likely: the model does not strongly depend on window content at *any* length, so varying W from 5 to 50 has little effect for the same reason that zeroing the context has little effect. The window-size ablation and the context-permutation controls (Section 4.4) are two views of the same underlying phenomenon, and we no longer present the former as evidence of short-range temporal structure specifically.

4.24. β Sensitivity and Training Convergence

Table 31 and Figure 12 report results for $\beta \in \{1, 2, 4, 6\}$ with $W = 20$ fixed.

Table 31. β sensitivity ($W = 20$ fixed). AUC variation is at most 0.0003 across all β values; FPR variation is below 0.002. The bold row indicates the selected configuration ($\beta = 2$).

β	BoT-IoT		IoT-23		ToN-IoT	
	AUC	**FPR**	**AUC**	**FPR**	**AUC**	**FPR**
1.0	0.9996	0.0068	0.9881	0.0560	0.9474	0.1317
2.0	**0.9996**	**0.0065**	**0.9879**	**0.0560**	**0.9471**	**0.1299**
4.0	0.9996	0.0065	0.9881	0.0563	0.9474	0.1308
6.0	0.9996	0.0065	0.9881	0.0563	0.9474	0.1308

Bold row indicates the selected configuration ($\beta = 2$) retained for consistency with the per-flow β-VAE baseline [5].

β has no measurable effect on performance because the KL divergence collapses to approximately 10^{-5} within the first 10 training epochs and stays there (Figure 13): once $D_{\mathrm{KL}} \approx 0$, the β multiplier scales a near-zero term and has no bearing on the gradient. This is the same collapse that, combined with Section 4.5's context-blindness finding, supports

our reading that z carries little information about its input—which is consistent with Lucas et al. [22]'s analysis of posterior collapse in linearised VAEs (Section 2.4).

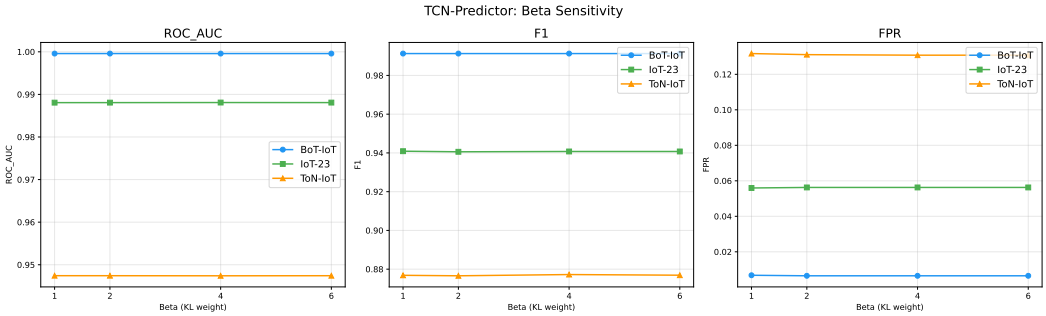


Figure 12. β sensitivity sweep: ROC-AUC, F1-score, and FPR vs. $\beta \in \{1, 2, 4, 6\}$ on all three datasets ($W = 20$ fixed). All curves are essentially flat: AUC varies by at most 0.0003 and FPR by less than 0.002.

Posterior and inference diagnostics. We evaluated additional latent statistics directly on all three test sets. The mean total KL (summing the 16 per-dimension terms for each flow) was only 5.81×10^{-4} on BoT-IoT, 3.89×10^{-4} on IoT-23, and 3.92×10^{-4} on ToN-IoT; the complete per-dimension means and the 2.5/25/50/75/97.5 percentiles of both μ and $\log \sigma^2$ are supplied in `results/latent_and_bootstrap_diagnostics.json`. Canonical inference uses one random latent sample per forward pass. Across 50 repeated samples of each flow, the mean within-flow score standard deviation was 0.0151–0.0157, and the mean coefficient of variation was only 1.16–1.26%. Decoding the posterior mean, using one sample, or averaging 50 sampled scores produced closely similar FPRs: respectively, 0.0062/0.0560/0.1305, 0.0059/0.0547/0.1335, and 0.0062/0.0563/0.1323 across BoT-IoT/IoT-23/ToN-IoT. These diagnostics confirm that sampling choice is not the source of the reported cross-dataset pattern and is consistent with a near-context-invariant predictor.

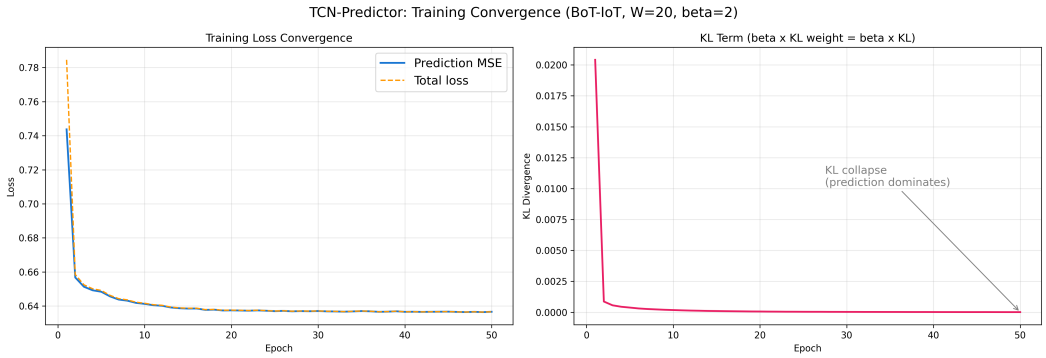


Figure 13. Training curves on BoT-IoT benign flows ($W = 20$, $\beta = 2$, 50 epochs). Left: prediction MSE drops over the first 10 epochs and levels off by epoch 30. Right: KL divergence collapses to $\approx 10^{-5}$ within the first 10 epochs and stays flat, consistent with the context-invariance documented in Section 4.5.

4.25. Inference Latency

We benchmark TCN-Pred's per-flow inference latency in single-sample streaming mode, matching the protocol used for the β-VAE baseline [5]. Each arriving flow is processed immediately upon receipt: the model maintains a rolling context buffer of the $W = 20$ preceding flows, constructs the context tensor, executes the full TCN forward pass, and computes the anomaly score. All measurements use the BoT-IoT test set on CPU without GPU acceleration, with 200 warm-up flows excluded from timing statistics.

TCN-Pred processes each flow in 5.15 ms (194 flows/s), compared with 0.16 ms (6213 flows/s) for the β-VAE. The originally reported 32$\times$ latency difference should be

read with the caveat above: Section 4.26's same-process, repeated-block measurement of both models finds a smaller (≈8.9×) ratio, so the 32× figure likely overstates the true gap, though the qualitative conclusion (TCN-Pred is substantially slower) holds under both measurements. The additional computation inherent to the temporal architecture is the same either way: each arrival requires constructing a $(1, W, F)$ context tensor from the rolling buffer, executing three causal TCN residual blocks, and performing the reparameterised forward pass operations absent from the per-flow β-VAE, which processes each flow as an independent vector through a two-layer MLP.

Hardware and software environment. All training and inference reported in this paper (including every ablation and control experiment) were executed on a single machine with an 11th Gen Intel® Core™ i5-1145G7 CPU (4 physical cores/8 logical threads; 2.60 GHz base clock) and 32 GB of RAM, running Windows 11. No GPU was used at any stage. Models are implemented in PyTorch 2.9 (CPU build) with the intra-op thread pool fixed at 4 threads; all floating-point operations use standard 32-bit precision (float32). The latency figures in Table 32 are the mean and 95th-percentile over all post-warm-up single-sample forward passes in one measurement run; we did not repeat the latency benchmark across multiple runs, so run-to-run measurement variance is not characterised and should be treated as a limitation of the timing claim. Mini-batch processing is expected to improve throughput by amortising the fixed per-call overhead of the TCN forward pass across multiple windows; we have not measured batched throughput in this study, so any proportional-scaling claim would be an untested expectation rather than a reported result.

Table 32. Per-flow inference latency in single-sample streaming mode (CPU only; BoT-IoT test set). β-VAE processes each flow independently as a single vector; TCN-Pred maintains a rolling context buffer of $W = 20$ preceding flows and executes a three-block causal TCN encoder on each arrival. Mean and 95th-percentile latency are reported over all timed samples after warm-up.

Model	Mean (µ s/Flow)	95th pct (µ s)	Throughput (Flows/s)
β-VAE	161	236	6213
TCN-Pred ($W = 20$)	5152	7371	194

β-VAE figures from the streaming evaluation in [5] are a single, unrepeated measurement run on a codebase and (as far as we can confirm from this repository alone) possibly different hardware than the TCN-Pred figures in this table; we cannot fully verify that the two rows were measured under identical conditions. Given Section 4.5, note that TCN-Pred's latency cost buys detection performance statistically indistinguishable from a context-blind constant predictor that would cost a fraction of this latency; Section 5 discusses the practical implication. **Section 4.26 reports a properly controlled comparison**, with every model measured in the same process, back-to-back, under an identical documented protocol, and this should be treated as the more trustworthy latency comparison; the 32× figure below is retained only as the originally reported, less rigorously controlled estimate.

Feature-extraction timing boundary. These latency figures begin with an already materialised 20-feature flow vector; they exclude packet capture, bidirectional flow assembly, CICFlowMeter execution, Zeek TLS parsing, and the upstream record join. Those stages cannot be benchmarked from the supplied artefacts because no PCAPs, executable extraction configuration, or join logs survive in this repository. Timing operations on the final NumPy arrays would not measure feature extraction, so we report this boundary explicitly and do not claim end-to-end gateway throughput.

We do not claim that TCN-Pred's measured 194 flows/s single-threaded CPU throughput (Table 32) is universally sufficient for "typical" IoT gateway deployments; no such typical rate is established or cited here. We instead sketch two illustrative, order-of-magnitude *scenarios*, not measurements from a specific deployment we instrumented and not verified rates drawn from a cited source: a consumer smart-home hub connecting on the order of 10–50 devices, each establishing at most a handful of new flows per minute during active use, would generate on the order of 1–10 new flows per second in aggregate; a small

industrial gateway polling a few dozen PLCs or sensors on short cycles might generate on the order of 10–50 new flows per second. Under these two illustrative scenarios only, the upper end is roughly 4–20× below TCN-Pred's measured throughput. Whether this margin is adequate for any real deployment depends on factors we have not measured and that vary by installation: the actual number of connected devices, their real flow-creation rate (which can spike well above steady-state averages during scans, updates, or bursty traffic); the deployment hardware (weaker embedded CPUs than the one used for our measurements would erode this margin); and, most importantly, whether the detector sits at a single edge gateway (the scenario these estimates describe) or at a larger aggregation point serving hundreds of devices or enterprise/ISP-scale traffic, where per-flow cost would need to be reduced (e.g., via batching, GPU acceleration, or the smaller-window configurations in Table 30) or the detector would need to run on a sample of traffic rather than every flow. Given Section 4.5, the unconditional mean baseline achieves statistically indistinguishable FPR/F1/AUC values at a fraction of this cost (no TCN forward pass at all; a single vector subtraction), and this is the more honest recommendation for a deployment that only needs the accuracy profile reported here.

4.26. Model Footprint and Deployment Cost

We quantify the "compact inference footprint" claim with parameter counts, model size in MBs, and peak memory at $W = 5$ vs. $W = 20$, rather than leave it as a bare assertion. `src/model_footprint.py` measures, in one process, back-to-back under an identical protocol, four variants: the per-flow β-VAE, TCN-Pred at the canonical $W = 20$, TCN-Pred at $W = 5$, and the context-blind constant predictor (Section 4.5). Protocol: CPU-only, PyTorch, float32 precision, `torch.set_num_threads(1)` (single-threaded, for a reproducible measurement—unlike the default multi-threaded timing behind Table 32, which is one reason the two tables' TCN-Pred numbers differ), batch size 1 (streaming scenario), 200 warm-up flows discarded and 2000 flows timed per block, 5 independently-warmed-up repeated blocks per model (run-to-run variation reported as the standard deviation across the 5 block means), on the same machine described above (11th Gen Intel Core i5-1145G7, 4 physical cores/8 logical threads, 32 GB RAM, Windows 11, PyTorch 2.9 CPU build)—unlike Table 32, where the β-VAE row's hardware provenance could not be fully confirmed. No $W = 5$ checkpoint was trained for this paper (only $W = 20$ is the evaluated, reported detector); $W = 5$'s parameter count and checkpoint size are architecture-only measurements from a freshly instantiated model of that configuration, stated as such rather than attributed to a trained checkpoint that does not exist.

Measured back-to-back under this controlled protocol, TCN-Pred ($W = 20$) is ≈8.9× slower than β-VAE (2315 µ s vs. 260.9 µ s) and ≈330× slower than the constant predictor (2315 µ s vs. 7.0 µ s)—which is substantial in both cases, though smaller than the originally reported 32× figure in Table 32, which compared measurements from two different sources/runs rather than a matched, single-process protocol (see that table's note). Reducing W from 20 to 5 saves almost nothing (Table 33's note), so window-size reduction is not a viable way to close TCN-Pred's latency gap. Combined with Section 4.5's finding that TCN-Pred, its LSTM/GRU analogues, and the context-blind constant predictor are statistically indistinguishable on every accuracy metric measured, we do not find evidence that TCN-Pred's additional latency and memory cost, at any tested window size, buys a measured practical benefit over the much cheaper context-blind alternative.

Table 33. Measured deployment footprint (CPU, single-threaded, batch = 1, mean of 5 repeated timing blocks of 2000 flows each after 200-flow warm-up).

Model	Params	Ckpt (KB)	Mean Lat. (μ s)	Peak RSS Δ (MB)	Thpt. (Flows/s)
β-VAE	25,012	103.1	260.9 ± 86.2	1.84	3833
TCN-Pred ($W = 20$)	60,116	255.6	2315.0 ± 93.1	0.97	432
TCN-Pred ($W = 5$)	60,116	255.8	2138.7 ± 376.2	0.10	468
Constant predictor	20 (mean vector)	0.08	7.0 ± 0.2	0.08	143,060

Latency: mean ± SD of the 5 block-mean latencies (run-to-run variation), not the within-block sample SD (which is larger; see `results/model_footprint.json` for both). TCN-Pred's parameter count and checkpoint size are identical at $W = 5$ and $W = 20$ (the TCN encoder's convolutional weights are shared across time steps and pooled to a fixed-size vector regardless of context length – only compute, not parameter count, depends on W). Reducing W by 4× (20→5) reduces mean latency by only ≈8% (2315→2139 μ s), indicating fixed per-call/Python-interpreter overhead, not the convolution FLOPs, dominates TCN-Pred's cost in this single-flow streaming regime; this means a smaller window is not an effective way to close the gap to the β-VAE or the constant predictor. The constant predictor is not a neural network: it has no encoder and no decoder, and no checkpoint file is produced by training this paper's pipeline; its 20 "parameters" are a single stored per-feature mean vector, and its 0.08 KB "checkpoint size" is that vector's raw float32 storage, not a serialised model file.

5. Discussion

5.1. Why Five Ablations Were Necessary

Each of the five negative results in Sections 4.3–4.6 tested a mechanism that was, on its own, a reasonable explanation for the FPR gap in Table 6. Target exclusion is exactly what we designed TCN-Pred to test. Context sensitivity is the property any genuinely sequential model should have. Decoder shape and training-window density are the two most obvious remaining architectural differences once the first two are ruled out. Had we stopped after confirming any single one of these—for instance, had the training-density ablation in Section 4.6 come back showing a large effect, which, on priors, it plausibly could have—we would have published a different, still-wrong explanation. The actual cause (Section 4.7) was not on our list of candidate mechanisms until the pattern of five consecutive null results left threshold calibration as the only remaining place to look. We report the full sequence, not just the final answer, because we think the sequence itself is the more transferable lesson: a single comparison between two "successful" and "unsuccessful" models, however carefully matched on paper, does not by itself establish which of their many differences is responsible, and mechanisms that sound right can survive a first check only to fail a second and third.

5.2. A General Hazard: Aggregation-Granularity Mismatch in Fixed-Threshold Calibration

The mechanism in Section 4.7 is not specific to TCN-VAEs or to network intrusion detection. Any fixed-threshold anomaly detector that (a) trains or calibrates using a score that has been aggregated (averaged, summed, and pooled) over multiple samples and (b) deploys using a score computed on a single sample or a differently-sized aggregate is exposed to the same effect: the calibration statistic will be systematically less variable than the deployment statistic. *The size of that reduction depends on the dependence structure of the averaged errors.* Under independence or weak dependence, the standard deviation shrinks by approximately the square root of the aggregation-size ratio, as observed empirically in this study (Section 4.7: a measured ratio of 4.46 against a $\sqrt{20} = 4.47$ prediction). When the errors being averaged within a window are positively correlated—for example, because consecutive flows share session-level features, or because a single attack episode spans many consecutive flows in the window—the reduction is smaller than $\sqrt{W}$, and in the limit of perfect correlation, the calibration statistic is no less variable than the deployment statistic at all, eliminating the mismatch entirely. Negative correlation would instead shrink the calibration statistic by *more* than $\sqrt{W}$, widening the effect. The $\sqrt{W}$ figure we report is,

therefore, best read as being consistent with approximately independent per-flow errors in this particular feature space, training regime, and (benign-only) calibration data, not as a universal constant; datasets or architectures with stronger within-window temporal correlation should expect a different ratio, and we recommend measuring it empirically rather than assuming $\sqrt{W}$. A threshold rule such as $\mu + k\sigma$ inherits whatever the actual mismatch turns out to be, silently producing an over-tight (or, in the reverse case, over-loose) operating point with no visible symptom other than the FPR itself, since ROC-AUC and other threshold-independent metrics are completely insensitive to it (Table 17). We recommend that any paper reporting a fixed threshold explicitly state the aggregation granularity of both the calibration statistic and the deployment score and verify they match before attributing a difference in fixed-threshold performance across models to architecture.

5.3. TCN-Pred's Real Behaviour

Section 4.5 showed that TCN-Pred, its LSTM/GRU analogues, and a context-blind constant predictor are statistically indistinguishable on every metric we measured, and Section 4.4 showed that TCN-Pred's own score barely moves when its context is destroyed. Taken together with the collapsed KL divergence reported in Section 4.24, the most defensible description of TCN-Pred is that it has learned an approximately context-invariant estimate of the benign-flow distribution and scores by squared distance from it—which is functionally close to the unconditional mean baseline, not a genuine sequential predictor. This does not make TCN-Pred's reported numbers wrong; it means the numbers do not support the causal-modelling explanation we originally gave for them, and a simpler, cheaper model achieves the same operational outcome (Section 4.25).

We do not know why gradient descent converged to this near-constant solution rather than a genuinely predictive one. One plausible account, consistent with the persistence and moving average baselines performing at chance (Table 14), is that individual IoT flow feature vectors in this feature space are not smoothly autocorrelated enough for a context-conditioned prediction to out-perform the context-independent mean under an MSE objective, so MSE-minimising training finds the mean. We did not test this account directly and flag it as the most likely, but unconfirmed, explanation.

5.4. Contamination as a Hypothesis, Not a Confirmed Mechanism

We no longer claim that the reconstruction-based TCN-VAE suffers from target/context contamination in the sense originally proposed: a shared latent code being distorted by co-occurring attack flows and inflating benign flows' reconstruction error. That specific mechanism, if present at all, is dominated by the calibration-granularity effect quantified in Section 4.7, which alone accounts for 89–97% of the reported FPR gap. We cannot rule out a smaller residual contamination effect superimposed on the granularity mismatch, but we have no evidence for one, and none of the five ablations in Sections 4.3–4.6 detected it. Framing the original hypothesis as confirmed was not supported by the evidence we now have.

5.5. What Does Survive?

Several of the original findings hold up under the stronger tests applied in this paper and are, we believe, genuine contributions independent of the reframing above: (i) the fixed source-domain threshold still out-performs Robust MAD and EMA adaptation even under clean, uncontaminated calibration (Section 4.13), though the margin and the reasons differ from the original claim; (ii) FPR genuinely remains stable across a realistic range of attack prevalence once contexts are properly reconstructed rather than reweighted (Section 4.14); (iii) unsupervised dimensionality reduction to 20 features (variance thresholding and PCA) destroys most of the usable signal, while the full 30-feature space remains informative, and

the labelled ANOVA–MI stage mainly trades a small amount of FPR for a large gain in recall over using all features unselected (Section 4.20); therefore, feature *selection* matters less than feature *reduction method* matters, which is itself an important, if less flattering, finding about where this line of work's performance actually comes from; and (iv) TLS metadata is not necessary for the reported ranking performance (Section 4.21). We no longer read this as supporting the encrypted-traffic framing of this study; Section 4.19 explains why the proportion of genuinely TLS-encrypted flows cannot be recovered from the artefacts available to us, so the more defensible reading is that TLS metadata is not necessary for the reported ranking performance, indicating that the results primarily reflect payload-independent flow statistics rather than a TLS-specific detection mechanism.

Point (iii) above is a direct consequence of how this pipeline is built, and we restate it plainly here because it qualifies every result in this Discussion: the pipeline as a whole is *supervised source-domain feature selection followed by benign-only detector training*, not a fully unsupervised system. Only the detector-training stage is benign-only/unsupervised; the feature-selection stage that determines what the detector can see is fitted on BoT-IoT attack labels.

5.6. Limitations

Data provenance. As detailed in Section 3.2, the shared per-dataset files carry no timestamp, capture, or device identifier, so several aspects of the original sequence-construction protocol (chronological order, capture boundaries, and train/test split granularity) cannot be independently verified from the artefacts available to us.

Source-population representativeness cannot be verified. Section 3.4 explains that no larger source file, sampling record, or seed exists for any of the three datasets, so the selected subsets' attack-family, protocol, and flow-statistic distributions cannot be compared against their true source populations.

TLS-observed proportion cannot be recovered. Section 4.19 shows that the four TLS-metadata columns were already numerically transformed upstream of every artefact available to us; we cannot state what fraction of flows in this study carried a genuine TLS handshake.

Classical-baseline provenance is not confirmed like-for-like. The provenance note in Section 4.9 explains that we cannot independently re-verify ref. [5]'s own split/seed/hyperparameters, and that Section 3.3's row-duplication finding raises a specific, unresolved possibility that the two studies' BoT-IoT partitions are not row-for-row identical.

mRMR stability protocol is lighter than ANOVA–MI's. Section 4.18 uses 50 bootstrap resamples at reduced size for mRMR versus 100 full-size resamples for ANOVA–MI for documented computational-feasibility reasons; the two stability estimates should not be read as directly comparable in magnitude.

Single feature-selection pipeline family. The label-free comparison (Section 4.20) tests only variance thresholding and PCA; other unsupervised criteria (e.g., autoencoder reconstruction-based selection and mutual information among features) were not tested and might close some of the gap to ANOVA–MI.

Row-level bootstrap independence, and what the cluster-level check does and does not establish. The paired bootstrap intervals in Section 4.10 resample individual evaluated flows with replacement, which formally assumes independent rows—an assumption Section 3.3 shows does not strictly hold, since each provided file reduces to only ≈6000 genuinely distinct flows duplicated 5.5–5.7 times on average. We tested this directly with a cluster-level (block) bootstrap (`src/cluster_bootstrap_sensitivity.py`, Section 4.10), which found interval widths within 0.1–0.9% of the row-level widths for the target-exposure FPR-difference statistic on IoT-23 and ToN-IoT. This is reassuring for

that specific statistic but is not evidence that row-level bootstrapping is generally safe in this setting: the surviving duplicate clusters among valid benign evaluation flows are small on average (≈1.7 rows/cluster after exclusions), which mechanically limits how different block resampling can be from row resampling. Statistics computed over a less filtered flow set or a dataset where more clusters survive intact could show a larger row vs. cluster discrepancy that this check would not detect. We therefore still recommend that cluster-aware resampling be treated as the more defensible default for this kind of duplicated data, and we report the row-level/cluster-level agreement found here as specific to the target-exposure comparison, not as a general property of every interval in this paper.

Residual contamination cannot be fully excluded. Section 4.7 explains the great majority of the observed FPR gap, but a small residual effect from the originally hypothesised mechanism cannot be ruled out with the precision of the experiments run here.

TCN-Det/TCN-Pred gap remains only partly explained. Section 4.8's matched-Gaussian-noise experiment rules out our initial hypothesis (that fixed reparameterisation-style noise alone widens σ_{train} enough to explain the gap). The five-seed sample-disjoint calibration check in Section 4.10 lowers TCN-Det FPR materially but leaves it above TCN-Pred and with much higher variance. Calibration optimism, therefore, explains part of the gap, but the remaining optimisation/stability difference is unresolved.

Benchmark datasets. All three datasets were collected under controlled laboratory conditions. BoT-IoT uses synthetic traffic; real-world enterprise or industrial IoT networks may exhibit greater flow diversity not captured in these benchmarks.

Balanced evaluation setup. The 50%/50% test split is a controlled benchmark condition; Section 4.14 addresses more realistic prevalence levels, but only within the same three benchmark datasets.

Single-machine timing. Latency and footprint figures (Sections 4.25 and 4.26) come from repeated measurement blocks on a single CPU (one machine and one architecture); no cross-hardware or embedded/ARM IoT-gateway measurement was performed.

Multi-seed coverage. Five-seed reliability (Section 4.10) was conducted for the TCN-family models central to this paper's diagnostic claims (TCN-Pred, the target-included variant, and TCN-Det) and, separately, for the recalibrated TCN-VAE (Section 4.7), the per-flow β-VAE, and the principal feature-selection comparison (Section 4.17). The classical baselines (RF, SVM, IF, and AE) remain single-seed results inherited from [5]; their comparisons remain descriptive because identical historical rows and preprocessing provenance cannot be independently established from this repository.

6. Conclusions

We set out to demonstrate that excluding a target flow from a reconstruction-based TCN-VAE's encoder prevents a contamination effect and explains why our causal predictor, TCN-Pred, achieves a far lower cross-dataset FPR (0.65–13%) than TCN-VAE (22–65%). We tested that explanation with controlled ablations rather than asserting it, and every one of five independent tests—target-exposure ablation, context-permutation controls, a context-blind baseline, decoder-shape ablation, and training-density ablation—falsified it. The actual mechanism, confirmed directly on the original, unmodified trained models, is a scoring-granularity mismatch: TCN-VAE's threshold was calibrated on per-window errors averaged over 20 flows (standard deviation 0.0496) and then applied to per-flow errors with a standard deviation $\sqrt{20} \approx 4.47\times$ higher (measured ratio: 4.46). Recalibrating the identical trained model at matching granularity, with no retraining, drops the FPR from 22.7/47.6/64.6% to 0.65/5.0/12.5% on BoT-IoT, IoT-23, and ToN-IoT, closing 89–97% of the gap we originally attributed to the architecture.

We also found, independently, that TCN-Pred itself is not doing what we designed it to do: shuffling, reversing, zeroing, or mean-replacing its context changes its FPR by at most 0.003, and a context-blind constant predictor matches its FPR and F1 to three decimal places on all three datasets. Combined with the already-reported collapse of the KL divergence to $\approx 10^{-5}$, the most defensible reading is that TCN-Pred has learned a context-invariant estimate of the benign-flow distribution, not a genuinely sequential one.

We think the paper's most useful contribution is, therefore, not the original causal-prediction result, but the diagnostic method that overturned it: a five-part ablation battery—target-exposure, context-permutation, a context-blind control, decoder-shape, and training-density—that any paper proposing a "temporal" or "causal" anomaly detector could and, we would argue, should run before attributing fixed-threshold performance differences to architecture, together with an explicit check that calibration and deployment scores share the same aggregation granularity. Several findings survive this scrutiny and remain genuine: a fixed source-domain threshold still out-performs naive adaptive alternatives under clean calibration; the FPR remains stable across the tested attack-prevalence levels in our controlled stream-reconstruction sensitivity analysis, in which contexts and predictions are recomputed at each prevalence level; TLS metadata is not required for the reported ranking performance; and the labelled ANOVA–MI feature-selection stage—not the choice of temporal architecture—accounts for most of the reported cross-dataset performance. We reiterate that this makes the overall pipeline *supervised source-domain feature selection followed by benign-only detector training*, which is not a fully unsupervised system; only the detector-training stage is benign-only. Future work should extend the granularity check to other temporal anomaly detectors in the literature, test whether an autoencoder-based or mutual-information-based label-free feature-selection criterion can close the gap to ANOVA–MI documented in Section 4.20, and investigate why gradient descent converges to a near-constant solution on this task rather than a genuinely context-conditioned one.

Finally, we deliberately narrow what these results are evidence for. TCN-Pred behaves as a context-invariant, effectively context-blind scorer rather than a genuinely temporal or sequential detector (Section 4.4), so this study does not validate temporal modelling as the source of its cross-dataset performance. Moreover, it does not validate a detector specifically tuned to encrypted traffic: the TLS-metadata columns available to us were already numerically transformed upstream of every artefact we could inspect, so the genuinely-TLS-encrypted proportion of the evaluated flows cannot be recovered (Section 4.19), and TLS metadata is in any case not required for the reported ranking performance (Section 4.21). The result that we expect to generalise beyond these three benchmarks is methodological, not architectural or deployment-specific: verify that a fixed threshold's calibration statistic and its deployment-time score share the same aggregation granularity before attributing a performance difference between two anomaly detectors to their architecture.

Author Contributions: Conceptualisation, M.N., R.U. and M.H.; methodology, M.N.; software, M.N.; validation, M.N., R.U. and M.H.; formal analysis, M.N.; investigation, M.N.; data curation, M.N.; writing—original draft preparation, M.N.; writing—review and editing, M.N., R.U. and M.H.; supervision, R.U. and M.H.; project administration, R.U. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The original BoT-IoT, IoT-23, and ToN-IoT datasets are available from their respective repositories [26–28]. This study analyses the processed experimental subsets described

in Table 3, not the raw published captures, and does not redistribute the source or processed traffic files. A minimal, data-free reproducibility package is supplied as supplementary material and contains the preprocessing and feature-selection code, model-training and evaluation scripts, calibration and ablation scripts, duplication and cluster-bootstrap analyses, anonymised per-row duplicate-cluster assignments, model configurations, hyperparameters, random seeds, environment specification, a manuscript-to-result manifest, and machine-readable results underlying the reported tables and statistical analyses. The cluster mapping contains dataset-local row identifiers, labels, split membership, and DBSCAN cluster identifiers, but no traffic feature values. Model checkpoints and processed feature arrays are available from the corresponding author (b01654699@studentmail.uws.ac.uk) on reasonable request. No original selected-row manifest, timestamp, capture identifier, or device identifier survives in the artefacts available to the authors. Consequently, the source-population sampling and capture chronology of the processed subsets cannot be reconstructed; Sections 3.2 and 3.4 document this limitation. A permanent archive DOI will be added to this statement when the versioned package is deposited.

Conflicts of Interest: The authors declare no conflict of interest.

Appendix A. ANOVA–MI Selected Features

The ANOVA–MI pipeline is fitted on BoT-IoT training labels and selects the top 20 features from the 30-column preprocessed feature matrix. The matrix comprises 26 bidirectional flow statistics produced by CICFlowMeter (v4.0; Canadian Institute for Cybersecurity, University of New Brunswick) followed by 4 TLS-metadata fields extracted by Zeek (v6.0; The Zeek Project) from `ssl.log` and `ja3.log`, as described in Section 3. Feature names follow the standard CICFlowMeter v4.0 and Zeek v6.0 column-naming conventions. Table A1 lists the 20 selected features in descending combined ANOVA + MI score order. The same feature set is applied unchanged to IoT-23 and ToN-IoT, preserving the no-recalibration protocol. Section 4.21 evaluates TCN-Pred with the two TLS-metadata features (ranks 13 and 19 below) removed.

Table A1. Top-20 features selected by the ANOVA–MI pipeline (descending combined ANOVA + MI score). Column indices are zero-based positions in the 30-column preprocessed feature matrix; indices 0–25 are CICFlowMeter v4.0 bidirectional flow statistics, and indices 26–29 are Zeek v6.0 TLS-metadata fields. Indices are verified from `results/selected_feature_indices_anova_mi.json`.

Rank	Index	Feature Name	Category
1	9	`bwd_pkt_len_max`	Packet Length Statistics
2	5	`fwd_pkt_len_max`	Packet Length Statistics
3	10	`bwd_pkt_len_min`	Packet Length Statistics
4	2	`tot_bwd_pkts`	Packet Volume Statistics
5	4	`totlen_bwd_pkts`	Packet Volume Statistics
6	6	`fwd_pkt_len_min`	Packet Length Statistics
7	8	`fwd_pkt_len_std`	Packet Length Statistics
8	7	`fwd_pkt_len_mean`	Packet Length Statistics
9	1	`tot_fwd_pkts`	Packet Volume Statistics
10	3	`totlen_fwd_pkts`	Packet Volume Statistics
11	11	`bwd_pkt_len_mean`	Packet Length Statistics
12	0	`flow_duration`	Flow Duration and Timing
13	26	`tls_version`	TLS Metadata
14	18	`flow_iat_min`	Flow Duration and Timing
15	15	`flow_iat_mean`	Flow Duration and Timing
16	13	`flow_byts_s`	Flow Rate Statistics
17	17	`flow_iat_max`	Flow Duration and Timing
18	22	`fwd_iat_max`	Flow Duration and Timing
19	27	`tls_cipher`	TLS Metadata
20	21	`fwd_iat_std`	Flow Duration and Timing

All 12 top-ranked features (indices 0–11) are CICFlowMeter packet-length and packet-volume statistics, reflecting their high combined discriminative power on the BoT-IoT source domain. TLS version and cypher suite (Zeek indices 26 and 27) appear at ranks 13 and 19; JA3 and JA3S fingerprint fields (indices 28 and 29) do not appear in the top 20. Section 4.21 shows that removing ranks 13 and 19 leaves ranking performance unchanged or marginally improved.

References

1. Anderson, B.; McGrew, D. Identifying encrypted malware traffic with contextual flow data. In Proceedings of the ACM Workshop on Artificial Intelligence and Security (AISec), Vienna, Austria, 28 October 2016; pp. 35–46. [CrossRef]
2. Rezaei, S.; Liu, X. A survey on TLS-encrypted malware network traffic analysis applicable to security operations centers. *Appl. Sci.* **2022**, *12*, 155. [CrossRef]
3. Chandola, V.; Banerjee, A.; Kumar, V. Anomaly detection: A survey. *ACM Comput. Surv.* **2009**, *41*, 15. [CrossRef]
4. Lopez-Martin, M.; Carro, B.; Sanchez-Esguevillas, A.; Lloret, J. Conditional variational autoencoder for prediction and feature recovery applied to intrusion detection in IoT. *Sensors* **2017**, *17*, 1967. [CrossRef] [PubMed]
5. Nouman, M.; Ujjan, R.; Hassanu, M. TLS-Aware Anomaly Detection for Encrypted IoT Traffic Using a β-Variational Autoencoder with ANOVA–Mutual Information Feature Selection. *Future Internet* **2026**, *18*, 310. [CrossRef]
6. Bai, S.; Kolter, J.Z.; Koltun, V. An empirical evaluation of generic convolutional and recurrent networks for sequence modeling. *arXiv* **2018**, arXiv:1803.01271.
7. Alaghbari, K.A.; Lim, H.S.; Saad, M.H.M.; Yong, Y.S. Deep autoencoder-based integrated model for anomaly detection and efficient feature extraction in IoT networks. *Internet Things* **2023**, *4*, 345–365. [CrossRef]
8. Hou, Y.; He, R.; Dong, J.; Yang, Y.; Ma, W. IoT anomaly detection based on autoencoder and Bayesian Gaussian mixture model. *Electronics* **2022**, *11*, 3287. [CrossRef]
9. Ayad, A.G.; El-Gayar, M.M.; Hikal, N.A.; Sakr, N.A. Efficient real-time anomaly detection in IoT networks using one-class autoencoder and deep neural network. *Electronics* **2025**, *14*, 104. [CrossRef]
10. Berraadi, O.; Gibet Tani, H.; Ben Ahmed, M. Enhancing security and privacy in IoT data streams: Real-time anomaly detection for threat mitigation in traffic management. *Comput. Sci. Math. Forum* **2025**, *10*, 8. [CrossRef]
11. Cantone, M.; Marrocco, C.; Bria, A. Machine learning in network intrusion detection: A cross-dataset generalization study. *IEEE Access* **2024**, *12*, 144489–144508. [CrossRef]
12. Wang, H.; Hu, P.; Gao, M.; Jiang, Y.; Wang, J.; Chen, T. An intrusion detection method combining variational auto-encoder and generative adversarial networks. *Comput. Netw.* **2024**, *253*, 110724. [CrossRef]
13. Higgins, I.; Matthey, L.; Pal, A.; Burgess, C.; Glorot, X.; Botvinick, M.; Mohamed, S.; Lerchner, A. β-VAE: Learning basic visual concepts with a constrained variational framework. In *Proceedings of ICLR 2017*; OpenReview: Alameda, CA, USA , 2017.
14. Ahmad, Z.; Khan, A.S.; Nisar, K.; Haider, I.; Hassan, R.; Haque, M.R.; Tarmizi, S.; Rodrigues, J.J.P.C. Anomaly detection using deep neural network for IoT architecture. *Appl. Sci.* **2021**, *11*, 7050. [CrossRef]
15. Owoh, N.; Riley, J.; Ashawa, M.; Hosseinzadeh, S.; Philip, A.; Osamor, J. An adaptive temporal convolutional network autoencoder for malicious data detection in mobile crowd sensing. *Sensors* **2024**, *24*, 2353. [CrossRef] [PubMed]
16. Hundman, K.; Constantinou, V.; Laporte, C.; Colwell, I.; Soderstrom, T. Detecting spacecraft anomalies using LSTMs and nonparametric dynamic thresholding. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*; ACM: New York, NY, USA, 2018; pp. 387–395. [CrossRef]
17. Munir, M.; Siddiqui, S.A.; Dengel, A.; Ahmed, S. DeepAnT: A deep learning approach for unsupervised anomaly detection in time series. *IEEE Access* **2019**, *7*, 1991–2005. [CrossRef]
18. Ji, I.H.; Lee, J.H.; Kang, M.J.; Park, W.J.; Jeon, S.H.; Seo, J.T. Artificial intelligence-based anomaly detection technology over encrypted traffic: A systematic literature review. *Sensors* **2024**, *24*, 898. [CrossRef] [PubMed]
19. Su, Y.; Zhao, Y.; Niu, C.; Liu, R.; Sun, W.; Pei, D. Robust anomaly detection for multivariate time series through stochastic recurrent neural network. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*; ACM: New York, NY, USA, 2019; pp. 2828–2837.
20. Audibert, J.; Michiardi, P.; Guyard, F.; Marti, S.; Zuluaga, M.A. USAD: Unsupervised anomaly detection on multivariate time series. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*; ACM: New York, NY, USA, 2020; pp. 3395–3404.
21. Bowman, S.R.; Vilnis, L.; Vinyals, O.; Dai, A.M.; Jozefowicz, R.; Bengio, S. Generating sentences from a continuous space. In *Proceedings of the 20th SIGNLL Conference on Computational Natural Language Learning (CoNLL)*; ACL: Berlin, Germany, 2016; pp. 10–21.
22. Lucas, J.; Tucker, G.; Grosse, R.B.; Norouzi, M. Don't blame the ELBO! A linear VAE perspective on posterior collapse. In *Advances in Neural Information Processing Systems 32 (NeurIPS 2019)*; Curran Associates: Red Hook, NY, USA, 2019.
23. Vovk, V.; Gammerman, A.; Shafer, G. *Algorithmic Learning in a Random World*; Springer: New York, NY, USA, 2005.
24. Gibbs, I.; Candès, E. Adaptive conformal inference under distribution shift. In *Advances in Neural Information Processing Systems 34 (NeurIPS 2021)*; Curran Associates: Red Hook, NY, USA, 2021.
25. Sharafaldin, I.; Lashkari, A.H.; Ghorbani, A.A. Toward generating a new intrusion detection dataset and intrusion traffic characterization. In *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*; SciTePress: Setubal, Portugal, 2018; pp. 108–116.

26. Koroniotis, N.; Moustafa, N.; Sitnikova, E.; Turnbull, B. Towards the development of realistic botnet dataset in the Internet of Things for network forensic analytics: Bot-IoT dataset. *Future Gener. Comput. Syst.* **2019**, *100*, 779–796. [CrossRef]
27. Garcia, S.; Parmisano, A.; Erquiaga, M.J. *IoT-23: A Labeled Dataset with Malicious and Benign IoT Network Traffic*, Version 1.0.0; Zenodo: Geneva, Switzerland, 2020. [CrossRef]
28. Moustafa, N. A new distributed architecture for evaluating AI-based security systems at the edge: Network TON_IoT datasets. *Sustain. Cities Soc.* **2021**, *72*, 102994. [CrossRef]
29. The Zeek Project. Zeek Network Security Monitor (Version 6.0). 2023. Available online: https://zeek.org (accessed on 22 May 2026).
30. Althouse, J.; Atkinson, J.; Atkins, J. TLS Fingerprinting with JA3 and JA3S. Salesforce Engineering Blog. 2019. Available online: https://engineering.salesforce.com/tls-fingerprinting-with-ja3-and-ja3s-247362855967 (accessed on 22 May 2026).
31. Amiri, F.; Rezaei Yousefi, M.M.; Lucas, C.; Shakery, A.; Yazdani, N. Mutual information-based feature selection for intrusion detection systems. *J. Netw. Comput. Appl.* **2011**, *34*, 1184–1199. [CrossRef]
32. Al-Sarem, M.; Saeed, F.; Alkhammash, E.H.; Alghamdi, N.S. An aggregated mutual information based feature selection with machine learning methods for enhancing IoT botnet attack detection. *Sensors* **2022**, *22*, 185. [CrossRef] [PubMed]
33. Peng, H.; Long, F.; Ding, C. Feature selection based on mutual information: Criteria of max-dependency, max-relevance, and min-redundancy. *IEEE Trans. Pattern Anal. Mach. Intell.* **2005**, *27*, 1226–1238. [CrossRef] [PubMed]

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.